



National Research University Higher School of Economics

SUSkva

Alexandr Ponkratov, Dmitry Pugachev, Vasilii Silvestrov

2023-11-22

Contest (1)

template.cpp	29 lines
<pre>#include <bits/stdc++.h> using namespace std; #define all(x) (x).begin(), (x).end() #define rall(x) (x).rbegin(), (x).rend() using ll = long long; using ld = long double; using ull = unsigned long long; const char en = '\n'; mt19937 rnd(chrono::high_resolution_clock::now().time_since_epoch().count()); template <typename T> int sz(T x) { return (int)x.size(); } void solve() {} int32_t main() { #define LOCAL freopen("input.txt", "r", stdin); #else ios_base::sync_with_stdio(0); cin.tie(0); #endif solve(); return 0; }</pre>	

hash.sh	3 lines
<pre># Hashes a file, ignoring all whitespace and comments. Use for # verifying that code was correctly typed. cpp -dD -P -fpreprocessed tr -d '[:space:]' md5sum cut -c-6</pre>	

CMakeLists.txt	8 lines
<pre>set(CMAKE_CXX_FLAGS "\${CMAKE_CXX_FLAGS} -Wall -Wshadow -g - fsanitize=address,undefined,bounds -D_GLIBCXX_DEBUG") file(GLOB APP_SOURCES \${CMAKE_CURRENT_SOURCE_DIR}/*.cpp) foreach(sourcefile \${APP_SOURCES}) file(RELATIVE_PATH filename \${CMAKE_CURRENT_SOURCE_DIR} \${ sourcefile}) string(REPLACE ".cpp" "" file \${filename}) add_executable(\${file} \${sourcefile}) endforeach(sourcefile \${APP_SOURCES})</pre>	

C++ (2)

gp-hashtable.cpp	e44914, 11 lines
Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).	
<pre>#include <ext/pb_ds/assoc_container.hpp> #include <ext/pb_ds/tree_policy.hpp> using namespace __gnu_pbds; const int RANDOM = chrono::high_resolution_clock::now().time_since_epoch().count();</pre>	

<pre>struct hasher { int operator()(int x) const { return x ^ RANDOM; } }; gp_hash_table<int, int, hasher> table;</pre>	
--	--

ordered-set.cpp	707789, 27 lines
Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. To get a map, change null_type. Time: $\mathcal{O}(\log(n))$	
<pre>#include <ext/pb_ds/assoc_container.hpp> #include <ext/pb_ds/tree_policy.hpp> using namespace __gnu_pbds; typedef tree<int, null_type, std::less<int>, rb_tree_tag, tree_order_statistics_node_update> ordered_set; int main() { ordered_set X; X.insert(1); X.insert(2); X.insert(4); X.insert(8); X.insert(16); cout << *X.find_by_order(1) << endl; // 2 cout << *X.find_by_order(2) << endl; // 4 cout << *X.find_by_order(4) << endl; // 16 cout << (end(X) == X.find_by_order(6)) << endl; // true cout << X.order_of_key(-5) << endl; // 0 cout << X.order_of_key(1) << endl; // 0 cout << X.order_of_key(3) << endl; // 2 cout << X.order_of_key(4) << endl; // 2 cout << X.order_of_key(400) << endl; // 5 }</pre>	

masks.cpp	b45elf, 11 lines
Description: Iterating through all the submasks and overmasks. Time: $\mathcal{O}(3^n)$	
<pre>for (int submask = mask;; submask = (submask - 1) & mask) { // use submask if (submask == 0) break; } for (int overmask = mask;; overmask = (overmask + 1) mask) { // use overmask if (overmask == maxmask) break; }</pre>	

Data structures (3)

dcp-offline.cpp	11870d, 146 lines
Description: DCP offline. Time: $\mathcal{O}(q \log^2(q))$	
<pre>struct Query { char type; int v, u; Query(char type) : type(type) {} Query(char type, int v, int u) : type(type), v(v), u(u) {} };</pre>	

<pre>struct DCP { int n, k, ans; // n - vertex, k - queries vector<int> par, rk; vector<pair<pii, int>> hist; // 0 - par, 1 - rk, 2 - ans; int qL, qR; pii edge; vector<vector<pii>> tree; vector<Query> qs; DCP(int _n, int _k) { n = ans = _n; par.resize(n); rk.resize(n, 1); for (int i = 0; i < n; ++i) par[i] = i; k = _k; tree.assign(4 * k, vector<pii>()); } int dsu_get(int v) { while (par[v] != v) v = par[v]; return v; } void dsu_unite(int a, int b) { a = dsu_get(a); b = dsu_get(b); if (a == b) return; if (rk[a] > rk[b]) swap(a, b); hist.emplace_back((pii){0, a}, par[a]); hist.emplace_back((pii){2, -1}, ans); par[a] = b; --ans; if (rk[a] == rk[b]) { hist.emplace_back((pii){1, b}, rk[b]); ++rk[b]; } } void dsu_unite(pii e) { dsu_unite(e.first, e.second); }</pre>	
<pre>void cancel(pair<pii, int> &el) { int &type = el.first.first; int &id = el.first.second; int &val = el.second; if (type == 0) { par[id] = val; } else if (type == 1) { rk[id] = val; } else if (type == 2) { ans = val; } }</pre>	
<pre>void add_edge(int _qL, int _qR, pii e) { // [L, R] qL = _qL; qR = _qR + 1; edge = e; add_edge_tree(1, 0, k); }</pre>	
<pre>void add_edge_tree(int v, int l, int r) { if (qL <= l && r <= qR) {</pre>	

```

        tree[v].emplace_back(edge);
        return;
    }
    int m = (1 + r) / 2, vL = 2 * v, vR = vL + 1;
    if (qL < m)
        add_edge_tree(vL, 1, m);
    if (m < qR)
        add_edge_tree(vR, m, r);
}

void go(vector<Query> &_qs) {
    qs = _qs;
    go_tree(1, 0, k);
}

void go_tree(int v, int l, int r) {
    int siz = sz(hist);
    for (auto &e : tree[v]) {
        dsu_unite(e);
    }
    if (l + 1 == r) {
        if (qs[l].type == '?') {
            cout << ans << en;
        }
    } else {
        int m = (1 + r) / 2, vL = 2 * v, vR = vL + 1;
        go_tree(vL, l, m);
        go_tree(vR, m, r);
    }
    while (sz(hist) > siz) {
        cancel(hist.back());
        hist.pop_back();
    }
}

};

void solve() {
    int n, k;
    cin >> n >> k;
    DCP dcp(n, k);
    set<pair<pii, int>> edges;
    vector<Query> qs;
    for (int i = 0; i < k; ++i) {
        char tp;
        cin >> tp;
        if (tp == '?') {
            qs.emplace_back(tp);
        } else {
            int v, u;
            cin >> v >> u;
            --v;
            --u;
            if (v > u)
                swap(v, u);
            qs.emplace_back(tp, v, u);
            if (tp == '+') {
                edges.emplace((pii){v, u}, i);
            } else {
                auto it = edges.lower_bound((pii){v, u}, 0);
                dcp.add_edge(it->second, i, it->first);
                edges.erase(it);
            }
        }
    }
    for (auto &e : edges) {
        dcp.add_edge(e.second, k - 1, e.first);
    }
    if (k)
        dcp.go(qs);
}

```

```

    }

linkcut.cpp
Description: Link-cut tree.
ddd3fe, 137 lines

struct Node {
    Node *ch[2] = {nullptr, nullptr};
    Node *par = nullptr;
    bool rev = false;
    int val, mn;
    int siz = 1;

    Node() {}

    Node(int val) : val(val), mn(val) {}
};

typedef Node *pnode;

int get_siz(pnode v) { return (v == nullptr ? 0 : v->siz); }

int get_min(pnode v) { return (v == nullptr ? INF : v->mn); }

void update(pnode v) {
    v->siz = 1 + get_siz(v->ch[0]) + get_siz(v->ch[1]);
    v->mn = min(v->val, min(get_min(v->ch[0]), get_min(v->ch[1])));
}

void push(pnode v) {
    if (v == nullptr || !v->rev) {
        return;
    }
    if (v->ch[0] != nullptr) {
        v->ch[0]->rev ^= 1;
    }
    if (v->ch[1] != nullptr) {
        v->ch[1]->rev ^= 1;
    }
    swap(v->ch[0], v->ch[1]);
    v->rev = 0;
}

bool is_root(pnode v) {
    return (v->par == nullptr || (v->par->ch[0] != v && v->par->ch[1] != v));
}

int child_num(pnode v) { return (v->par->ch[1] == v); }

void attach(pnode v, pnode p, int num) {
    if (v != nullptr) {
        v->par = p;
    }
    if (p != nullptr) {
        p->ch[num] = v;
    }
}

void rotate(pnode v) {
    int num = child_num(v);
    pnode p = v->par, vb = v->ch[num ^ 1];
    pnode g = (p == nullptr ? nullptr : p->par);
    if (g != nullptr) {
        if (!is_root(p)) {
            g->ch[child_num(p)] = v;
        }
    }
    v->par = g;
}

```

```

    attach(p, v, num ^ 1);
    attach(vb, p, num);
    update(p);
    update(v);
}

void splay(pnode v) {
    vector<pnode> st;
    pnode cur = v;
    st.emplace_back(cur);
    while (!is_root(cur)) {
        cur = cur->par;
        st.emplace_back(cur);
    }
    for (int i = sz(st) - 1; i >= 0; --i) {
        push(st[i]);
    }
    while (!is_root(v)) {
        if (!is_root(v->par)) {
            if (child_num(v) == child_num(v->par)) {
                rotate(v->par);
            } else {
                rotate(v);
            }
        }
        rotate(v);
    }
}

void expose(pnode v) {
    splay(v);
    v->ch[1] = nullptr;
    update(v);
    while (v->par != nullptr) {
        splay(v->par);
        attach(v, v->par, 1);
        update(v->par);
        splay(v);
    }
}

void make_root(pnode v) {
    expose(v);
    v->rev ^= 1;
}

void link(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    u->par = v;
}

void cut(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    push(u);
    u->ch[1] = nullptr;
    v->par = nullptr;
}

bool is_connected(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    if (is_root(v) && u != v) {
        return false;
    } else {
        return true;
    }
}
}

```

```
int get_min(pnode v, pnode u) {
    make_root(v);
    make_root(u);
    return get_min(u);
}
```

segtreebeats.cpp

Description: Segment Tree beats (TODO: WRITE OPERATIONS). 92635a, 238 lines

```
struct SegTree {
    struct Node {
        ll max, sec_max;
        int cnt_max;
        ll min, sec_min;
        int cnt_min;
        ll sum;
        ll push_add;
        ll push_eq;
    };

    int n, qL, qR;
    ll val, ans;

    vector<Node> tree;
    vector<ll> a;

    SegTree(int _n) {
        n = _n;
        tree.assign(4 * n, {0, -INFL, 1, 0, INFL, 1, 0, 0, -1});
    }

    void update_vertex(int v, int l, int r) {
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;

        tree[v].sum = tree[vL].sum + tree[vR].sum;

        tree[v].max = max(tree[vL].max, tree[vR].max);
        tree[v].sec_max = max(tree[vL].sec_max, tree[vR].sec_max);
        tree[v].cnt_max = 0;
        if (tree[vL].max == tree[v].max) {
            tree[v].cnt_max += tree[vL].cnt_max;
        } else {
            tree[v].sec_max = max(tree[v].sec_max, tree[vL].max);
        }
        if (tree[vR].max == tree[v].max) {
            tree[v].cnt_max += tree[vR].cnt_max;
        } else {
            tree[v].sec_max = max(tree[v].sec_max, tree[vR].max);
        }

        tree[v].min = min(tree[vL].min, tree[vR].min);
        tree[v].sec_min = min(tree[vL].sec_min, tree[vR].sec_min);
        tree[v].cnt_min = 0;
        if (tree[vL].min == tree[v].min) {
            tree[v].cnt_min += tree[vL].cnt_min;
        } else {
            tree[v].sec_min = min(tree[v].sec_min, tree[vL].min);
        }
        if (tree[vR].min == tree[v].min) {
            tree[v].cnt_min += tree[vR].cnt_min;
        } else {
            tree[v].sec_min = min(tree[v].sec_min, tree[vR].min);
        }
    }

    void recalc_eq(int v, int l, int r, ll cur) {
        tree[v].max = tree[v].min = tree[v].push_eq = cur;
        tree[v].sec_max = -INFL;
```

```
        tree[v].sec_min = INFL;
        tree[v].cnt_max = tree[v].cnt_min = r - 1;
        tree[v].sum = cur * (ll)(r - 1);
        tree[v].push_add = 0;
    }

    void recalc_add(int v, int l, int r, ll cur) {
        if (tree[v].min == tree[v].max) {
            recalc_eq(v, l, r, tree[v].max + cur);
            return;
        }
        tree[v].max += cur;
        if (tree[v].sec_max != -INFL) {
            tree[v].sec_max += cur;
        }

        tree[v].min += cur;
        if (tree[v].sec_min != INFL) {
            tree[v].sec_min += cur;
        }

        tree[v].sum += (ll)(r - 1) * cur;
        tree[v].push_add += cur;
    }

    void recalc_min(int v, int l, int r, ll cur) {
        if (tree[v].min >= cur) {
            recalc_eq(v, l, r, cur);
            return;
        }
        if (tree[v].max <= cur)
            return;
        if (tree[v].sec_min == tree[v].max) {
            tree[v].sec_min = cur;
        }
        tree[v].sum -= (ll)(tree[v].max - cur) * (ll)(tree[v].cnt_max);
        tree[v].max = cur;
    }

    void recalc_max(int v, int l, int r, ll cur) {
        if (tree[v].max <= cur) {
            recalc_eq(v, l, r, cur);
            return;
        }
        if (tree[v].min >= cur)
            return;
        if (tree[v].sec_max == tree[v].min) {
            tree[v].sec_max = cur;
        }
        tree[v].sum += (ll)(tree[v].max - cur) * (ll)(tree[v].cnt_max);
        tree[v].min = cur;
    }

    void push(int v, int l, int r) {
        if (l + 1 == r)
            return;
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        if (tree[v].push_eq != -1) {
            recalc_eq(vL, l, m, tree[v].push_eq);
            recalc_eq(vR, m, r, tree[v].push_eq);
            tree[v].push_eq = -1;
            return;
        }

        recalc_add(vL, l, m, tree[v].push_add);
        recalc_add(vR, m, r, tree[v].push_add);
        tree[v].push_add = 0;
```

```
        recalc_min(vL, l, m, tree[v].max);
        recalc_min(vR, m, r, tree[v].max);

        recalc_max(vL, l, m, tree[v].min);
        recalc_max(vR, m, r, tree[v].min);
    }

    void build(vector<ll> &a) {
        a = _a;
        build_tree(1, 0, n);
    }

    void build_tree(int v, int l, int r) {
        if (l + 1 == r) {
            tree[v] = {a[l], -INFL, 1, a[l], INFL, 1, 0, 0, -1};
            return;
        }
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        build_tree(vL, l, m);
        build_tree(vR, m, r);
        update_vertex(v, l, r);
    }

    void update_segment_min(int _qL, int _qR, ll _val) {
        qL = _qL;
        qR = _qR + 1;
        val = _val;
        update_segment_min_tree(1, 0, n);
    }

    void update_segment_min_tree(int v, int l, int r) {
        if (tree[v].max <= val)
            return;
        if (qL <= l && r <= qR && tree[v].sec_max < val) {
            recalc_min(v, l, r, val);
            return;
        }
        push(v, l, r);
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        if (qL < m)
            update_segment_min_tree(vL, l, m);
        if (m < qR)
            update_segment_min_tree(vR, m, r);
        update_vertex(v, l, r);
    }

    void update_segment_max(int _qL, int _qR, ll _val) {
        qL = _qL;
        qR = _qR + 1;
        val = _val;
        update_segment_max_tree(1, 0, n);
    }

    void update_segment_max_tree(int v, int l, int r) {
        if (tree[v].min >= val)
            return;
        if (qL <= l && r <= qR && tree[v].sec_min > val) {
            recalc_max(v, l, r, val);
            return;
        }
        push(v, l, r);
        int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
        if (qL < m)
            update_segment_max_tree(vL, l, m);
        if (m < qR)
            update_segment_max_tree(vR, m, r);
        update_vertex(v, l, r);
    }
}
```

```
void update_segment_add(int _qL, int _qR, ll _val) {
    qL = _qL;
    qR = _qR + 1;
    val = _val;
    update_segment_add_tree(1, 0, n);
}

void update_segment_add_tree(int v, int l, int r) {
    if (qL <= l && r <= qR) {
        recalc_add(v, l, r, val);
        return;
    }
    push(v, l, r);
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    if (qL < m)
        update_segment_add_tree(vL, l, m);
    if (m < qR)
        update_segment_add_tree(vR, m, r);
    update_vertex(v, l, r);
}

ll get(int _qL, int _qR) {
    qL = _qL;
    qR = _qR + 1;
    ans = 0;
    get_tree(1, 0, n);
    return ans;
}

void get_tree(int v, int l, int r) {
    if (qL <= l && r <= qR) {
        ans += tree[v].sum;
        return;
    }
    push(v, l, r);
    int m = (l + r) / 2, vL = 2 * v, vR = vL + 1;
    if (qL < m)
        get_tree(vL, l, m);
    if (m < qR)
        get_tree(vR, m, r);
}
};
```

cht.cpp

Description: Convex-hull trick.

7e474f, 48 lines

```
struct Line {
    ll k = 0, b = -INFL;
    Line() = default;
    Line(ll k, ll b) : k(k), b(b){};

    ld operator()(ld x) { return k * x + b; }
};

bool operator<(Line a, Line b) {
    return a.k < b.k || (a.k == b.k && a.b > b.b);
}

bool operator<(pair<Line, ld> a, pair<Line, ld> b) {
    return a.second < b.second;
}

struct CHT {
    vector<pair<Line, ld>> convex;

    void add(Line a) {
        while (!convex.empty() && a(convex.back().second) >
```

```
convex.back().first(convex.back().second)) {
        convex.pop_back();
    }
    ld xn = -INFL;
    if (!convex.empty()) {
        xn = (a.b - convex.back().first.b + 0.0) / (convex.back().first.k - a.k);
    }
    convex.push_back({a, xn});
}

CHT(vector<Line> lines) {
    sort(all(lines));
    for (int i = 0; i < (int)lines.size(); ) {
        int j = i;
        while (j < (int)lines.size() && lines[i].k == lines[j].k)
            ++j;
        add(lines[i]);
        i = j;
    }
}

ld get(ld x) {
    auto it = upper_bound(all(convex), pair<Line(), x>) -
        convex.begin();
    return convex[it - 1].first(x);
}
};
```

lichao.cpp

Description: Li-Chao tree.

107896, 64 lines

```
struct LiChao { // max
    struct Line {
        ll k = 0, b = -INFL;

        Line() = default;

        Line(ll k, ll b) : k(k), b(b){};

        ll operator()(ll x) { return k * x + b; }
    };

    struct Node {
        Node *l = nullptr, *r = nullptr;
        Line cur;

        Node() = default;
    };

    Node *root = nullptr;
    int n = 1e9 + 1;

    void make_node(Node *&v) {
        if (v == nullptr) {
            v = new Node();
        }
    }

    void add(Node *&v, int l, int r, Line cur) {
        make_node(v);
        int m = (l + r) / 2;
        if (cur(m) > v->cur(m)) {
            swap(cur, v->cur);
        }
        if (l + 1 == r) {
```

```
return;
    }
    if (cur(l) > v->cur(l)) {
        add(v->l, l, m, cur);
    } else {
        add(v->r, m, r, cur);
    }
}

void add(Line cur) { add(root, 0, n, cur); }

ll get(Node *v, int l, int r, int x) {
    if (v == nullptr) {
        return -INFL;
    }
    ll ans = v->cur(x);
    if (l + 1 == r) {
        return ans;
    }
    int m = (l + r) / 2;
    if (x < m) {
        ans = max(ans, get(v->l, l, m, x));
    } else {
        ans = max(ans, get(v->r, m, r, x));
    }
    return ans;
}

ll get(int x) { return get(root, 0, n, x); }
};
```

Graph (4)

dominator-tree.cpp

Description: Dominator tree.

154ba6, 157 lines

```
struct Edge {
    int from, to, id;

    Edge() = default;
    Edge(int from, int to, int id) : from(from), to(to), id(id) {}
};

struct DSU {
    int n;
    vector<int> par;
    vector<pii> mn;

    DSU() = default;
    DSU(int n) : n(n) {
        par.resize(n);
        mn.resize(n);
        init();
    }

    void init() {
        for (int i = 0; i < n; ++i) {
            par[i] = i;
            mn[i] = {INF, i};
        }
    }

    int get(int v) {
        if (par[v] == v) {
            return v;
        }
        int p = get(par[v]);
```

```

    //      mn[v] = min(mn[v], mn[par[v]]);
    if (mn[par[v]].first < mn[v].first) {
        mn[v] = mn[par[v]];
    }
    par[v] = p;
    return p;
}

void unite(int a, int b) { par[a] = b; }
};

struct DominatorTree {
    int n;
    vector<Edge> edges;
    vector<int> sdom, idom, tin, order, par, used, dp;
    vector<vector<int>> g, rg, queries;
    DSU dsu_sdom, dsu_idom;

    DominatorTree() = default;
    DominatorTree(int n) : n(n), dsu_sdom(n), dsu_idom(n) {
        sdom.resize(n, INF); // semi-dominator
        idom.resize(n, INF); // immediate dominator
        tin.resize(n, -1);
        par.resize(n);
        used.resize(n, 0);
        dp.resize(n, INF);
        g.resize(n);
        rg.resize(n);
        queries.resize(n);
    }

    void add_edge(int from, int to) { edges.emplace_back(from, to, sz(edges)); }

    void dfs(int v) {
        tin[v] = sz(order);
        order.emplace_back(v);
        for (auto i : g[v]) {
            const auto &e = edges[i];
            if (tin[e.to] == -1) {
                par[e.to] = v;
                dfs(e.to);
            }
        }
    }

    void dfs_idom(int v) {
        used[v] = 1;
        for (auto i : g[v]) {
            const auto &e = edges[i];
            if (!used[e.to]) {
                dfs_idom(e.to);
            }
        }
        for (auto u : queries[v]) {
            dsu_idom.get(u);
            dp[u] = dsu_idom.mn[u].second;
        }
        dsu_idom.mn[v] = {sdom[v], v};
        for (auto i : g[v]) {
            const auto &e = edges[i];
            if (par[e.to] == v) {
                dsu_idom.unite(e.to, v);
            }
        }
    }

    void build(int s) {
        for (int i = 0; i < sz(edges); ++i) {

```

```

            g[edges[i].from].emplace_back(i);
            rg[edges[i].to].emplace_back(i);
        }
        // reorder vertex
        dfs(s);

        // build sdom
        for (int _ = sz(order) - 1; _ >= 0; --_) {
            int v = order[_];
            if (v == s) {
                continue;
            }
            for (auto i : rg[v]) {
                const auto &e = edges[i];
                if (tin[e.from] == -1) {
                    continue;
                }
                if (tin[e.from] < tin[v]) {
                    sdom[v] = min(sdom[v], tin[e.from]);
                } else {
                    int u = dsu_sdom.get(e.from);
                    sdom[v] = min(sdom[v], dsu_sdom.mn[e.from].first);
                }
            }
            dsu_sdom.mn[v] = {sdom[v], v};
            for (auto i : g[v]) {
                const auto &e = edges[i];
                if (v == par[e.to]) {
                    dsu_sdom.unite(e.to, v);
                }
            }
        }

        // build queries for idoms
        for (int i = 0; i < n; ++i) {
            if (i == s || sdom[i] == INF || tin[i] == -1) {
                continue;
            }
            queries[order[sdom[i]]].emplace_back(i);
        }
        dfs_idom(s);

        // build idom
        idom[s] = tin[s];
        for (auto v : order) {
            if (v == s) {
                continue;
            }
            if (v == dp[v]) {
                idom[v] = sdom[v];
            } else {
                idom[v] = idom[dp[v]];
            }
        }
    }

    int get_idom(int v) { return (idom[v] == INF ? -1 : order[idom[v]]); }
};

```

eulerian-path.cpp

Description: Finds eulerian path.

687663, 16 lines

```

vector<vector<int>> g;
vector<bool> used;
vector<int> ed, tour;

```

```

void dfs(int v) {
    while (!g[v].empty()) {

```

```

        int u = g[v].back();
        g[v].pop_back();
        if (used[u]) {
            continue;
        }
        used[u] = 1;
        dfs(ed[u] ^ v);
    }
    tour.push_back(v);
}

```

hld.cpp

Description: Heavy-light decomposition.

Time: $\mathcal{O}(\log^2(n))$ per query

98d176, 88 lines

```

struct HLD {

    struct SegTree {};

    int n, T;
    SegTree st;
    vector<vector<int>> tree;
    vector<int> par, siz, tin, tout, head;
    ll ans;

    HLD(int _n) {
        n = _n;
        tree.resize(n);
        par.resize(n, -1);
        siz.resize(n, 0);
        tin.resize(n);
        tout.resize(n);
        head.resize(n);
        st = SegTree(n);
    }

    void add_edge(int v, int u) {
        tree[v].emplace_back(u);
        tree[u].emplace_back(v);
    }

    void build(int v = 0) {
        dfs_siz(v, v);
        T = 0;
        head[v] = v;
        dfs_hld(v, v);
    }

    void dfs_siz(int v, int p) {
        par[v] = p;
        siz[v] = 1;
        for (auto &u : tree[v]) {
            if (u != p) {
                dfs_siz(u, v);
                siz[v] += siz[u];
            }
        }
        for (int i = 0; i < sz(tree[v]); ++i) {
            int x = tree[v][0], u = tree[v][i];
            if (x == p || siz[u] > siz[x]) {
                swap(tree[v][0], tree[v][i]);
            }
        }
    }

    void dfs_hld(int v, int p) {
        tin[v] = T++;
        for (auto u : tree[v]) {
            if (u == p) {

```

```
        continue;
    }
    if (u == tree[v][0]) {
        head[u] = head[v];
    } else {
        head[u] = u;
    }
    dfs_hld(u, v);
}
tout[v] = T;
}

void update(int v, int val) { st.update_segment(tin[v], tin[v], val); }

bool is_anc(int v, int u) { return tin[v] <= tin[u] && tout[u] <= tout[v]; }

void go_up(int &v, int u) {
    while (!is_anc(head[v], u)) {
        ans = max(ans, st.get(tin[head[v]], tin[v]));
        v = par[head[v]];
    }
}

ll get(int v, int u) { // max on path
    ans = -INFL;
    go_up(v, u);
    go_up(u, v);
    if (!is_anc(v, u)) {
        swap(v, u);
    }
    ans = max(ans, st.get(tin[v], tin[u]));
    return ans;
}
};
```

kuhn.cpp

Description: Kuhn's algorithm.

b95c30, 27 lines

```
vector<vector<int>>> g;
vector<int> used, mt;
int timer = 1;
bool dfs(int v) {
    if (used[v] == timer) {
        return false;
    }
    used[v] = timer;
    for (auto u : g[v]) {
        if (mt[u] == -1) {
            mt[u] = v;
            return true;
        }
    }
    for (auto u : g[v]) {
        if (dfs(mt[u])) {
            mt[u] = v;
            return true;
        }
    }
    return false;
}
for (int i = 0; i < n; ++i) {
    if (dfs(i)) {
        ++timer;
    }
}
```

maxflow.cpp

Description: Dinic's algorithm

Time: $\mathcal{O}(V^2E)$

608498, 116 lines

```
struct MaxFlow {
    struct Edge {
        ll flow, cap;
        int to, id;

        Edge() {}

        Edge(ll flow, ll cap, int to, int id)
            : flow(flow), cap(cap), to(to), id(id) {}
    };

    int n;
    vector<vector<Edge>>> g;
    vector<int> d, head, used;
    ll max_cap;
    int s, t;

    MaxFlow() {}

    MaxFlow(int _n) {
        n = _n;
        g.resize(n);
    }

    void add_edge(int from, int to, ll cap) {
        g[from].emplace_back(0, cap, to, sz(g[to]));
        g[to].emplace_back(0, 0, from, sz(g[from]) - 1);
    }

    bool bfs() {
        d.assign(n, INF);
        d[s] = 0;
        queue<int> q;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (auto e : g[v]) {
                if (d[e.to] == INF && e.cap - e.flow >= max_cap) {
                    d[e.to] = d[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return d[t] != INF;
    }

    ll dfs(int v, ll cur_flow) {
        if (v == t) {
            return cur_flow;
        }
        for (; head[v] < sz(g[v]); ++head[v]) {
            auto &e = g[v][head[v]];
            if (e.cap - e.flow >= max_cap && d[v] + 1 == d[e.to]) {
                ll new_flow = dfs(e.to, min(cur_flow, e.cap - e.flow));
                if (new_flow) {
                    e.flow += new_flow;
                    g[e.to][e.id].flow -= new_flow;
                    return new_flow;
                }
            }
        }
        return 0;
    }
};
```

```
ll find_max_flow(int _s, int _t) {
    s = _s;
    t = _t;
    ll res = 0;
    for (int k = 30; k >= 0; --k) {
        max_cap = (1 << k);
        while (bfs()) {
            head.assign(n, 0);
            ll flow = 0;
            do {
                flow = dfs(s, INFL);
                res += flow;
            } while (flow);
        }
    }
    return res;
}

ll dfs_const_flow(int v, ll cur_flow) {
    used[v] = 1;
    if (v == t) {
        return cur_flow;
    }
    for (auto &e : g[v]) {
        if (!used[e.to] && e.cap - e.flow > 0) {
            ll new_flow = dfs_const_flow(e.to, min(cur_flow, e.cap - e.flow));
            if (new_flow) {
                e.flow += new_flow;
                g[e.to][e.id].flow -= new_flow;
                return new_flow;
            }
        }
    }
    return 0;
}

bool find_const_flow(int _s, int _t, ll F) {
    s = _s;
    t = _t;
    ll res = 0, flow = 0;
    max_cap = F;
    do {
        used.assign(n, 0);
        flow = dfs_const_flow(s, INF);
        res += flow;
    } while (flow && res < F);
    return res == F;
}

ll get_edge_flow(int v, int id) { return g[v][id].flow; }
};

mincost.cpp
Description: Similar to the Edmonds-Karp.
Time:  $\mathcal{O}(VE + FE \log(V))$ 
126796, 88 lines
struct MinCostMaxFlow {
    struct Edge {
        ll flow, cap, price;
        int to, id;

        Edge() {}

        Edge(ll flow, ll cap, ll price, int to, int id)
            : flow(flow), cap(cap), price(price), to(to), id(id) {}
    };

    int n;
```

```
int s, t;
ll ans;
vector<vector<Edge>> g;
vector<int> d;
vector<ll> add_f;
vector<pii> par;

MinCostMaxFlow() {}

MinCostMaxFlow(int _n) {
    n = _n;
    g.resize(n);
}

void add_edge(int from, int to, ll cap, ll price) {
    g[from].emplace_back(0, cap, price, to, sz(g[to]));
    g[to].emplace_back(0, 0, -price, from, sz(g[from]) - 1);
}

ll get_edge_flow(int v, int id) { return g[v][id].flow; }

void FB() {
    d.assign(n, INF);
    add_f.assign(n, 0);
    par.assign(n, {-1, -1});
    d[s] = 0;
    add_f[0] = INF;
    queue<int> q;
    q.push(s);
    vector<int> used(n, 0);
    used[s] = 1;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        used[v] = 0;
        for (int i = 0; i < sz(g[v]); ++i) {
            auto &e = g[v][i];
            if (e.flow < e.cap && d[e.to] > d[v] + e.price) {
                d[e.to] = d[v] + e.price;
                add_f[e.to] = min(add_f[v], e.cap - e.flow);
                par[e.to] = {v, i};
                if (!used[e.to]) {
                    q.push(e.to);
                    used[e.to] = 1;
                }
            }
        }
    }
}

void push_flow(ll flow) {
    int cur = t;
    while (cur != s) {
        int prev = par[cur].first, id = par[cur].second;
        g[prev][id].flow += flow;
        g[cur][g[prev][id].id].flow -= flow;
        ans += g[prev][id].price * flow;
        cur = prev;
    }
}

ll min_cost_max_flow(int _s, int _t) {
    ans = 0;
    s = _s;
    t = _t;
    while (true) {
        FB();
        ll flow = add_f[t];
        if (flow == 0) {

```

```
break;
        }
        push_flow(flow);
    }
    return ans;
}
};
```

Strings (5)

z-func.cpp

Description: Z-function.

11739d, 18 lines

```
vector<int> z_function(string &s) {
    int n = (int)s.size();
    vector<int> z(n, 0);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i <= r) {
            z[i] = min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            r = i + z[i] - 1;
            l = i;
        }
    }
    return z;
}
```

prefix-func.cpp

Description: Prefix function.

8cb94c, 14 lines

```
vector<int> prefix_function(string &s) {
    int n = (int)s.size();
    vector<int> p(n, 0);
    for (int i = 1; i < n; i++) {
        int cur = p[i - 1];
        while (s[i] != s[cur] && cur > 0) {
            cur = p[cur - 1];
        }
        if (s[i] == s[cur]) {
            p[i] = cur + 1;
        }
    }
    return p;
}
```

sufarray.cpp

Description: Suffix array and LCP.

Time: $\mathcal{O}(n \log(n))$ and $\mathcal{O}(n)$

efc54c, 71 lines

```
int mod(int x, int m) {
    if (x < 0)
        x += m;
    if (x >= m)
        x -= m;
    return x;
}

vector<int> sufux_array(string s) {
    s += '$';
    int n = sz(s);
    vector<int> arr(n), narr(n), head(n), c(n), nc(n);
    for (int i = 0; i < n; ++i) {
        arr[i] = i;

```

```
    }
    sort(all(arr), [&](int x, int y) { return s[x] < s[y]; });
    int cl = 0;
    c[arr[0]] = cl;
    head[0] = 0;
    for (int i = 1; i < n; ++i) {
        if (s[arr[i]] != s[arr[i - 1]]) {
            head[++cl] = i;
        }
        c[arr[i]] = cl;
    }
    for (int k = 1; k < n && cl < n; k *= 2) {
        for (int i = 0; i < n; i++) {
            int j = mod(arr[i] - k, n);
            narr[head[c[j]]++] = j;
        }
        cl = 0;
        head[0] = 0;
        pii prev = {c[narr[0]], c[mod(narr[0] + k, n)]};
        for (int i = 1; i < n; i++) {
            pii cur = {c[narr[i]], c[mod(narr[i] + k, n)]};
            if (cur != prev) {
                head[++cl] = i;
            }
            nc[narr[i]] = cl;
            prev = cur;
        }
        swap(arr, narr);
        swap(c, nc);
    }
    // returns suffix array without $
    return vector<int>(arr.begin() + 1, arr.end());
}

vector<int> build_lcp(string &s, vector<int> &suf) {
    int n = sz(s);
    vector<int> lcp(n - 1), order(n);
    for (int i = 0; i < n; ++i) {
        order[suf[i]] = i;
    }
    int l = 0;
    for (int i = 0; i < n; ++i) {
        int id = order[i];
        if (id + 1 == n) {
            l = 0;
            continue;
        }
        int j = suf[id + 1];
        if (l)
            --l;
        while (max(i + l, j + l) < n && s[i + l] == s[j + l]) {
            ++l;
        }
        lcp[id] = l;
    }
    return lcp;
}
```

sufavtomat.cpp

Description: Suffix automaton.

23f122, 51 lines

```
struct Node {
    int go[26];
    int suf, prev, term, len;
};

Node() {
    for (auto &i : go) {
        i = -1;
    }
}
```

```
len = 0;
suf = -1;
prev = -1;
term = 0;
}
};

vector<Node> automat;

int add(int a, int ch) {
    int b = sz(automat);
    automat.emplace_back();
    automat[b].prev = a;
    automat[b].suf = 0;
    automat[b].len = automat[a].len + 1;
    for (; a != -1; a = automat[a].suf) {
        if (automat[a].go[ch] == -1) {
            automat[a].go[ch] = b;
            continue;
        }
        int c = automat[a].go[ch];
        if (automat[c].prev == a) {
            automat[b].suf = c;
            break;
        }
    }
    int d = sz(automat);
    automat.emplace_back();
    automat[d].suf = automat[c].suf;
    automat[d].len = automat[a].len + 1;
    automat[c].suf = d;
    automat[b].suf = d;
    automat[d].prev = a;
    for (int i = 0; i < 26; ++i) {
        automat[d].go[i] = automat[c].go[i];
    }
    for (; a != -1 && automat[a].go[ch] == c; a = automat[a].suf) {
        automat[a].go[ch] = d;
    }
    break;
}
// returns id of the added vertex
return b;
}
```

korasik.cpp

Description: Aho–Corasick algorithm.

9682e1, 63 lines

```
struct Node {
    int par;
    int par_c;
    int go[26];
    int term;
    int link;
    int super;
    int cnt;
};

const int N = 1e6 + 5;
int ptr = 1;
Node trie[N];

void add(string s) {
    int cur = 1;
    for (auto &q : s) {
        int c = q - 'a';
        if (trie[cur].go[c]) {
            cur = trie[cur].go[c];
        } else {
```

```
++ptr;
        trie[cur].go[c] = ptr;
        trie[ptr].par = cur;
        trie[ptr].par_c = c;
        cur = ptr;
    }
    trie[cur].cnt++;
}

void build() {
    queue<int> q;
    q.push(1);
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        if (v != 1) {
            if (trie[v].par == 1) {
                trie[v].link = trie[v].super = 1;
            } else {
                trie[v].link = trie[trie[trie[v].par].link].go[trie[v].par_c];
                trie[v].super =
                    (trie[trie[v].link].cnt ? trie[v].link : trie[trie[v].link].super);
            }
        }
        trie[v].cnt += trie[trie[v].link].cnt;
        for (int c = 0; c < 26; c++) {
            if (trie[v].go[c]) {
                q.push(trie[v].go[c]);
            } else {
                if (v == 1) {
                    trie[v].go[c] = 1;
                } else {
                    trie[v].go[c] = trie[trie[v].link].go[c];
                }
            }
        }
    }
}

trie[1].link = trie[1].super = trie[1].par = 1;
build();
```

manacher.cpp

Description: Manacher's algorithm.

ab7999, 28 lines

```
vector<int> manacher_odd(string s) {
    int n = (int)s.size();
    vector<int> d(n, 1);
    int l = 0, r = 0;
    for (int i = 1; i < n; i++) {
        if (i < r)
            d[i] = min(r - i + 1, d[l + r - i]);
        while (i - d[i] >= 0 && i + d[i] < n && s[i - d[i]] == s[i + d[i]])
            d[i]++;
        if (i + d[i] - 1 > r)
            l = i - d[i] + 1, r = i + d[i] - 1;
    }
    return d;
}

vector<int> manacher_even(string s) {
    int n = (int)s.size();
    vector<int> d(n, 0);
    int l = -1, r = -1;
    for (int i = 0; i < n - 1; i++) {
        if (i < r)
```

```
        d[i] = min(r - i, d[l + r - i - 1]);
        while (i - d[i] >= 0 && i + d[i] + 1 < n && s[i - d[i]] == s[i + d[i] + 1])
            d[i]++;
        if (i + d[i] > r)
            l = i - d[i] + 1, r = i + d[i];
    }
    return d;
}
```

Math (6)

fft.cpp

Description: FFT.

5f442d, 119 lines

```
constexpr int P = 998244353;
using i64 = long long;

// assume -P <= x < 2P
int norm(int x) {
    if (x < 0) {
        x += P;
    }
    if (x >= P) {
        x -= P;
    }
    return x;
}

template <class T> T power(T a, int b) {
    T res = 1;
    for (; b; b /= 2, a *= a) {
        if (b % 2) {
            res *= a;
        }
    }
    return res;
}

struct Z {
    int x;
    Z(int x = 0) : x(norm(x)) {}
    int val() const { return x; }
    Z operator-() const { return Z(norm(P - x)); }
    Z inv() const {
        assert(x != 0);
        return power(*this, P - 2);
    }
    Z &operator*=(const Z &rhs) {
        x = i64(x) * rhs.x % P;
        return *this;
    }
    Z &operator+=(const Z &rhs) {
        x = norm(x + rhs.x);
        return *this;
    }
    Z &operator-=(const Z &rhs) {
        x = norm(x - rhs.x);
        return *this;
    }
    Z &operator/=(const Z &rhs) { return *this *= rhs.inv(); }
    friend Z operator*(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res *= rhs;
        return res;
    }
    friend Z operator+(const Z &lhs, const Z &rhs) {
        Z res = lhs;
```

```
        res += rhs;
        return res;
    }
    friend Z operator-(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res -= rhs;
        return res;
    }
    friend Z operator/(const Z &lhs, const Z &rhs) {
        Z res = lhs;
        res /= rhs;
        return res;
    }
};

std::vector<int> rev;
std::vector<Z> roots{0, 1};
void dft(std::vector<Z> &a) {
    int n = a.size();

    if (int(rev.size()) != n) {
        int k = __builtin_ctz(n) - 1;
        rev.resize(n);
        for (int i = 0; i < n; i++) {
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
        }
    }

    for (int i = 0; i < n; i++) {
        if (rev[i] < i) {
            std::swap(a[i], a[rev[i]]);
        }
    }
    if (int(roots.size()) < n) {
        int k = __builtin_ctz(roots.size());
        roots.resize(n);
        while ((1 << k) < n) {
            Z e = power(Z(3), (P - 1) >> (k + 1));
            for (int i = 1 << (k - 1); i < (1 << k); i++) {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * e;
            }
            k++;
        }
    }
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                Z u = a[i + j];
                Z v = a[i + j + k] * roots[k + j];
                a[i + j] = u + v;
                a[i + j + k] = u - v;
            }
        }
    }
}

void idft(std::vector<Z> &a) {
    int n = a.size();
    std::reverse(a.begin() + 1, a.end());
    dft(a);
    Z inv = (1 - P) / n;
    for (int i = 0; i < n; i++) {
        a[i] *= inv;
    }
}
```

factorization.cpp

Description: Pollard's rho algorithm.

Time: $\mathcal{O}(\sqrt[3]{n})$

68092c, 106 lines

```
ll binpow(ll a, ll n, ll mod) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        int128_t b = binpow(a, n / 2, mod);
        return (b * b) % mod;
    }
    return ((int128_t)a * binpow(a, n - 1, mod)) % mod;
}

constexpr int N = 1e7;
vector<int> pr, lp;

bool prime(ll n) {
    if (n <= N) {
        return binary_search(all(pr), n);
    }
    int iter = 60;
    int s = 0;
    ll d = n - 1;
    while (d % 2 == 0) {
        d /= 2;
        ++s;
    }
    auto test = [&](ll a) {
        if (binpow(a, d, n) == 1) {
            return true;
        }
        ll _2r = 1;
        for (int r = 0; r < s; ++r) {
            auto tmp = binpow(binpow(a, d, n), _2r, n);
            if (tmp == n - 1) {
                return true;
            }
            _2r *= 2;
        }
        return false;
    };
    for (int _ = 0; _ < iter; ++_) {
        ll a = uniform_int_distribution<ll>(1, n - 1)(rnd);
        if (!test(a)) {
            return false;
        }
    }
    return true;
}

ll f(ll x, ll n) { return ((int128_t)x * (int128_t)x + (int128_t)2) % n; }

ll pollard(ll n) {
    ll a = uniform_int_distribution<ll>(0, n - 1)(rnd);
    ll x = a, y = a, d = 1;
    constexpr int iter = 5e4;
    for (int _ = 0; _ < iter; ++_) {
        x = f(f(x, n), n);
        y = f(y, n);
        d = gcd(abs(x - y), n);
        if (d != 1 && d != n) {
            break;
        }
    }
    if (d == 1 || d == n) {
        pollard(n);
    }
}
```

```
    }
    return d;
}

vector<ll> res;

void factor(ll n) {
    if (n <= N) {
        while (true) {
            if (lp[n] == 0) {
                break;
            }
            res.push_back(lp[n]);
            n /= lp[n];
        }
        return;
    }
    if (prime(n)) {
        res.push_back(n);
        return;
    }
    ll d = pollard(n);
    factor(n / d);
    factor(d);
}

void solve() {
    ll n;
    cin >> n;
    lp.resize(N + 1);
    for (int i = 2; i <= N; ++i) {
        if (lp[i] == 0) {
            lp[i] = i;
            pr.push_back(i);
        }
        for (int j = 0; j < (int)pr.size() && pr[j] <= lp[i] && i * pr[j] <= N; ++j) {
            lp[i * pr[j]] = pr[j];
        }
    }
    factor(n);
}

crt.cpp
Description: Chinese Remainder Theorem.
812c50, 70 lines

struct Eq { // x = a (mod m)
    ll a, m;

    Eq(){};

    Eq(ll a, ll m) : a(a), m(m){};
};

ll binpow(ll a, ll n, ll m) {
    if (n == 0) {
        return 1;
    }
    if (n % 2 == 0) {
        int128_t b = binpow(a, n / 2, m);
        return (b * b) % m;
    }
    int128_t x = binpow(a, n - 1, m);
    return (a * x) % m;
}

ll binpow(ll a, ll n) {
    if (n == 0) {
```

```
        return 1;
    }
    if (n % 2 == 0) {
        ll b = binpow(a, n / 2);
        return b * b;
    }
    return a * binpow(a, n - 1);
}

ll phi(ll a) {
    ll d = 2, k = a;
    map<ll, int> cnt;
    while (d * d <= a) {
        if (k % d == 0) {
            k /= d;
            ++cnt[d];
        } else {
            ++d;
        }
    }
    if (k != 1) {
        ++cnt[k];
    }
    ll ans = 1;
    for (auto i : cnt) {
        ans *= binpow(i.first, i.second - 1) * (i.first - 1);
    }
    return ans;
}

ll gcd(ll a, ll b) { return std::gcd(abs(a), abs(b)); }

Eq solve(Eq ai, Eq bi) {
    if (ai.m == -1 || bi.m == -1) {
        return {0, -1};
    }
    ll a = ai.m, b = bi.m, c = ai.a - bi.a;
    ll d = ::gcd(a, b);
    a /= d;
    b /= d;
    if (c % d != 0) {
        return {0, -1};
    }
    c /= d;
    ll x = (((int128_t)-c * (int128_t)binpow(a, phi(b) - 1, b)) %
            b + b) % b;
    x = ((int128_t)ai.m * (int128_t)x + ai.a) % lcm(ai.m, bi.m);
    return {x, lcm(ai.m, bi.m)};
}
```

ext-gcd.cpp

Description: Extended GCD. 6e27a2, 72 lines

```
template <typename T> T extgcd(T a, T b, T &x, T &y) {
    if (a == 0) {
        x = 0;
        y = 1;
        return b;
    }
    T p = b / a;
    T g = extgcd(b - p * a, a, y, x);
    x -= p * y;
    return g;
}
```

```
template <typename T> bool diophantine(T a, T b, T c, T &x, T &y, T &g) {
    if (a == 0 && b == 0) {
        if (c == 0) {
```

```
        x = y = g = 0;
        return true;
    }
    return false;
}
if (a == 0) {
    if (c % b == 0) {
        x = 0;
        y = c / b;
        g = abs(b);
        return true;
    }
    return false;
}
if (b == 0) {
    if (c % a == 0) {
        x = c / a;
        y = 0;
        g = abs(a);
        return true;
    }
    return false;
}
g = extgcd(a, b, x, y);
if (c % g != 0) {
    return false;
}
T dx = c / a;
c -= dx * a;
T dy = c / b;
c -= dy * b;
x = dx + (T)((__int128)x * (c / g) % b);
y = dy + (T)((__int128)y * (c / g) % a);
g = abs(g);
return true;
// |x|, |y| <= max(|a|, |b|, |c|) [tested]
}
```

```
bool crt(long long k1, long long m1, long long k2, long long m2,
         long long &k,
         long long &m) {
    k1 %= m1;
    if (k1 < 0)
        k1 += m1;
    k2 %= m2;
    if (k2 < 0)
        k2 += m2;
    long long x, y, g;
    if (!diophantine(m1, -m2, k2 - k1, x, y, g)) {
        return false;
    }
    long long dx = m2 / g;
    long long delta = x / dx - (x % dx < 0);
    k = m1 * (x - dx * delta) + k1;
    m = m1 / g * m2;
    assert(0 <= k && k < m);
    return true;
}
```

gauss.cpp

Description: Gaussian Elimination. 0b7335, 58 lines

```
const double eps = 1e-9;
int Gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = (int)a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
    double det = 1;
    int rank = 0;
    for (int col = 0, row = 0; col < m && row < n; ++col) {
```

```
int mx = row;
for (int i = row; i < n; i++) {
    if (fabs(a[i][col]) > fabs(a[mx][col])) {
        mx = i;
    }
}
if (fabs(a[mx][col]) < eps) {
    det = 0;
    continue;
}
for (int i = col; i <= m; i++) {
    swap(a[row][i], a[mx][i]);
}
if (row != mx) {
    det = -det;
}
det *= a[row][col];
pos[col] = row;
for (int i = 0; i < n; i++) {
    if (i != row && fabs(a[i][col]) > eps) {
        double c = a[i][col] / a[row][col];
        for (int j = col; j <= m; j++) {
            a[i][j] -= a[row][j] * c;
        }
    }
}
++row;
++rank;
}
ans.assign(m, 0);
for (int i = 0; i < m; i++) {
    if (pos[i] != -1) {
        ans[i] = a[pos[i]][m] / a[pos[i]][i];
    }
}
for (int i = 0; i < n; i++) {
    double sum = 0;
    for (int j = 0; j < m; j++) {
        sum += ans[j] * a[i][j];
    }
    if (fabs(sum - a[i][m]) > eps) {
        return -1; // no solution
    }
}
for (int i = 0; i < m; i++) {
    if (pos[i] == -1) {
        return 2; // infinte solutions
    }
}
return 1; // unique solution
}
```

bin-gauss.cpp

Description: Gaussian Elimination modulo 2. n = number of equations, m = number of variables 39a607, 68 lines

```
int Gauss(int n, int m, vector<bitset<N>> a, bitset<N> &ans) {
    // reversing for lexicographically largest solution
    for (int i = 0; i < n; i++) {
        bitset<N> tmp;
        for (int j = 0; j < m; j++)
            tmp[j] = a[i][m - j - 1];
        tmp[m] = a[i][m];
        a[i] = tmp;
    }
    int rank = 0, det = 1;
    vector<int> pos(N, -1);
    for (int col = 0, row = 0; col < m && row < n; ++col) {
```

```
for (int i = row; i < n; ++i)
    if (a[i][col]) {
        mx = i;
        break;
    }
if (!a[mx][col]) {
    det = 0;
    continue;
}
swap(a[mx], a[row]);
if (row != mx) {
    det = (det == 0 ? 0 : 1);
}
det &= a[row][col];
pos[col] = row;
// forward elimination
for (int i = row + 1; i < n; ++i) {
    if (i != row && a[i][col]) {
        a[i] ^= a[row];
    }
}
++row, ++rank;
}
ans.reset();
// backward substitution
for (int i = m - 1; i >= 0; i--) {
    if (pos[i] == -1) {
        ans[i] = true;
    } else {
        int k = pos[i];
        for (int j = i + 1; j < m; j++)
            if (a[k][j])
                ans[i] = ans[i] ^ ans[j];
        ans[i] = ans[i] ^ a[k][m];
    }
}
for (int i = rank; i < n; ++i) {
    if (a[i][m]) {
        return -1; // no solution
    }
}
// reversing again beacuse we reversed earlier
bitset<N> tmp;
for (int j = 0; j < m; j++) {
    tmp[j] = ans[m - j - 1];
}
ans = tmp;
int free_var = 0;
for (int i = 0; i < m; ++i) {
    if (pos[i] == -1) {
        free_var++;
    }
}
return free_var; // has solution
}
```

mult-inv.cpp
Description: Calc multiplicative inverse.

93296c, 19 lines

```
int inv(int a, int m) {
    if (a == 1)
        return 1;
    return (1 - inv(m % a, a) * m) / a + m;
}

f[0] = 1;
for (int i = 1; i < N; i++) {
    f[i] = i * f[i - 1] % mod;
}
```

```
r[N - 1] = inv(f[N - 1]);
for (int i = N - 1; i >= 1; i--) {
    r[i - 1] = r[i] * i % mod;
}
// _____
r[1] = 1;
for (int i = 2; i < m; ++i) {
    r[i] = (m - (m / i) * r[m % i] % m) % m;
}
```

Geometry (7)

point.cpp
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

c2b847, 59 lines

```
template <typename T> struct point_t {
    T x, y;
    point_t() : x(0), y(0) {}
    point_t(T _x, T _y) : x(_x), y(_y) {}
    T len_sq() const { return x * x + y * y; }
    ld len() const { return sqrtl(len_sq()); }
    point_t operator*(T k) const { return {x * k, y * k}; }
    void operator*=(T k) {
        x *= k;
        y *= k;
    }
    point_t operator+(const point_t<T> &other) const {
        return {x + other.x, y + other.y};
    }
    point_t operator-(const point_t<T> &other) const {
        return {x - other.x, y - other.y};
    }
    bool operator==(const point_t<T> &other) const {
        return x == other.x && y == other.y;
    }
    bool operator!=(const point_t<T> &other) const { return !(*
        this == other); }
    T operator*(const point_t<T> &other) const {
        return x * other.x + y * other.y;
    } // dot product
    T operator%(const point_t<T> &other) const {
        return x * other.y - y * other.x;
    } // cross product
};
using Point = point_t<ll>;

template <typename T> istream &operator>>(istream &is, point_t<
    T> &vec) {
    is >> vec.x >> vec.y;
    return is;
}

template <typename T> ostream &operator<<(ostream &os, const
    point_t<T> &vec) {
    os << vec.x << ' ' << vec.y;
    return os;
}

bool cmp_vectors(Point a, Point b) {
    if ((a.y < 0 || (a.y == 0 && a.x < 0)) &&
        (b.y > 0 || (b.y == 0 && b.x > 0))) {
        return true;
    }
    if ((b.y < 0 || (b.y == 0 && b.x < 0)) &&
        (a.y > 0 || (a.y == 0 && a.x > 0))) {
        return false;
    }
}
```

```
return (a % b > 0 || (a % b == 0 && a.len_sq() < b.len_sq()))
    ;
}

int get_sign(ld x) {
    if (x < -EPS)
        return -1;
    if (EPS < x)
        return 1;
    return 0;
}
```

polygon.cpp
Description: Polygon functions

09e4ae, 100 lines

```
using Polygon = vector<Point>;

Polygon build_convex_hull(Polygon &a) {
    int n = sz(a);
    for (int i = 1; i < n; ++i) {
        if ((a[i].y < a[0].y) || (a[i].y == a[0].y && a[i].x < a
            [0].x))
            swap(a[0], a[i]);
    }
    sort(a.begin() + 1, a.end(), [&](Point A, Point B) {
        Point oa = A - a[0], ob = B - a[0];
        if ((oa % ob) == 0)
            return oa.len_sq() < ob.len_sq();
        return (oa % ob) > 0;
    });
    Polygon hull = {a[0]};
    for (int i = 1; i < n; ++i) {
        while (sz(hull) >= 2) {
            Point ab = hull[sz(hull) - 1] - hull[sz(hull) - 2],
                bp = a[i] - hull[sz(hull) - 1];
            if ((ab % bp) <= 0) {
                hull.pop_back();
            } else {
                break;
            }
        }
        hull.emplace_back(a[i]);
    }
    return hull;
}

ld area_of_polygon(Polygon &poly) {
    ll res = 0;
    int n = sz(poly);
    for (int i = 2; i < n; ++i) {
        Point ab = poly[i - 1] - poly[0], ac = poly[i] - poly[0];
        res += (ab % ac);
    }
    return abs(res) / 2;
}

ld perimeter_of_polygon(Polygon &poly) {
    ld res = 0;
    int n = sz(poly);
    for (int i = 0; i < n; ++i) {
        Point v = poly[(i + 1 == n ? 0 : i + 1)] - poly[i];
        res += v.len();
    }
    return res;
}

int point_in_polygon(const Point &p, const Polygon &poly) {
    // 0 - outside, 1 - inside, 2 - border;
    int n = sz(poly);
```

```
if (point_in_angle(p, poly[n - 1], poly[0], poly[1])) {
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int md = (l + r) / 2;
        if (point_in_angle(p, poly[md], poly[0], poly[1])) {
            r = md;
        } else {
            l = md;
        }
    }
    if (point_in_angle(p, poly[r], poly[0], poly[1]) &&
        point_in_angle(p, poly[0], poly[1], poly[r]) &&
        point_in_angle(p, poly[1], poly[r], poly[0])) {
        if (point_on_segment(p, poly[1], poly[r]) ||
            point_on_segment(p, poly[(l ? 1 - 1 : n - 1)], poly[l
                ]) ||
            point_on_segment(p, poly[r], poly[(r + 1 == n ? 0 : r
                + 1)])) {
            return 2;
        } else {
            return 1;
        }
    } else {
        return 0;
    }
} else {
    return 0;
}
}

int point_in_nonconvex_polygon(const Point &p, const Polygon &
    poly) {
    // 0 - outside, 1 - inside, 2 - border;
    int n = sz(poly);
    for (int i = 0; i < n; ++i) {
        if (point_on_segment(p, poly[i], poly[(i + 1) % n])) {
            return 2;
        }
    }
    ld s = 0.0;
    for (int i = 0; i < n; ++i) {
        Point pa = poly[i] - p, pb = poly[(i + 1 == n ? 0 : i + 1)]
            - p;
        s += angle_rad(pa, pb);
    }
    if (s >= PI || s <= -PI) {
        return 1;
    } else {
        return 0;
    }
}
```

angle.cpp

Description: Some stuff with angles.

76aeac, 16 lines

```
template <typename T> ld angle(const point_t<T> &a, const
    point_t<T> &b) {
    return fabs1(atan2(a % b, a * b) / PI * 180);
}

template <typename T> ld angle_rad(const point_t<T> &a, const
    point_t<T> &b) {
    return (atan2(a % b, a * b));
}

template <typename T> ld angle_rad(const point_t<T> &a) {
    return atan2(a.y, a.x);
}
```

```
template <typename T> point_t<ld> rotate(const point_t<T> &a,
    ld alpha) {
    return {a.x * cos(alpha) - a.y * sin(alpha),
        a.x * sin(alpha) + a.y * cos(alpha)};
}
```

distance.cpp

Description: Calc the distance between everything.

8a21fc, 78 lines

```
ld from_point_to_line(const Point &p, const Point &a,
    const Point &b) { // point p, line ab
    Point ba = a - b, ap = p - a;
    return fabs((ba % ap) / ba.len());
}

ld from_point_to_ray(const Point &p, const Point &a,
    const Point &b) { // point p, ray ab
    Point ba = a - b, ap = p - a, ab = b - a;
    if ((ab * ap) < 0) {
        return ap.len();
    } else {
        return fabs((ba % ap) / ba.len());
    }
}

ld from_point_to_segment(const Point &p, const Point &a,
    const Point &b) { // point p, segment
        ab
    Point ab = b - a, ap = p - a, bp = p - b, ba = a - b;
    if ((ab * ap) < 0) {
        return fabs1(ap.len());
    } else if ((ab * bp) > 0) {
        return fabs1(bp.len());
    } else {
        return fabs1((ba % ap) / ba.len());
    }
}

ld from_segment_to_segment(const Point &a, const Point &b,
    const Point &c,
        const Point &d) { // segment ab,
        segment cd
    if (segment_intersection(a, b, c, d)) {
        return 0;
    }
    return min({from_point_to_segment(a, c, d),
        from_point_to_segment(b, c, d),
        from_point_to_segment(c, a, b),
        from_point_to_segment(d, a, b)});
}

ld from_segment_to_ray(const Point &a, const Point &b, const
    Point &c,
        const Point &d) { // segment ab, ray cd
    if (ray_segment_intersection(c, d, a, b)) {
        return 0;
    }
    return min({from_point_to_ray(a, c, d), from_point_to_ray(b,
        c, d),
        from_point_to_segment(c, a, b)});
}

ld from_segment_to_line(const Point &a, const Point &b, const
    Point &c,
        const Point &d) { // segment ab, line
        cd
    Point cd = d - c, ca = a - c, cb = b - c;
    if (get_sign(cd % ca) * get_sign(cd % cb) <= 0) {
        return 0;
    }
}
```

```
}
return min(from_point_to_line(a, c, d), from_point_to_line(b,
    c, d));
}

ld from_ray_to_ray(const Point &a, const Point &b, const Point
    &c,
        const Point &d) { // ray ab, ray cd
    if (rays_intersection(a, b, c, d)) {
        return 0;
    }
    return min(from_point_to_ray(a, c, d), from_point_to_ray(c, a
        , b));
}

ld from_ray_to_line(const Point &a, const Point &b, const Point
    &c,
        const Point &d) { // ray ab, line cd
    if (line_ray_intersection(c, d, a, b)) {
        return 0;
    }
    return from_point_to_line(a, c, d);
}

ld from_line_to_line(const Point &a, const Point &b, const
    Point &c,
        const Point &d) { // line ab, line cd
    if (lines_intersection(a, b, c, d)) {
        return 0;
    }
    return from_point_to_line(a, c, d);
}
```

intersections.cpp

Description: Intersect everything.

59a62e, 120 lines

```
bool point_on_line(const Point &p, const Point &a,
    const Point &b) { // point p, line ab
    Point ab = b - a, ap = p - a;
    return (ab % ap) == 0;
}

bool point_on_ray(const Point &p, const Point &a,
    const Point &b) { // point p, ray ab
    Point ab = b - a, ap = p - a;
    return ((ab % ap) == 0 && (ab * ap) > 0) || (a == p);
}

bool point_on_segment(const Point &p, const Point &a,
    const Point &b) { // point p, segment ab
    Point ap = p - a, bp = p - b, ab = b - a;
    return (ap * bp) <= 0 && (ap % ab) == 0;
}

bool point_in_angle(const Point &p, const Point &a, const Point
    &o,
        const Point &b) { // point p, angle aob
    Point oa = a - o, ob = b - o, op = p - o;
    if ((oa % ob) < 0)
        swap(oa, ob);
    return ((oa % op) >= 0 && (ob % op) <= 0);
}

bool segment_intersection(const Point &a, const Point &b, const
    Point &c,
        const Point &d) { // segment ab,
        segment, cd
    Point ab = b - a, cd = d - c, ac = c - a, ad = d - a, cb = b
        - c, ca = a - c;
```

```

    if (get_sign((ab % ac)) * get_sign((ab % ad)) <= 0 &&
        get_sign((cd % ca)) * get_sign((cd % cb)) <= 0) {
        ll x1 = max(min(a.x, b.x), min(c.x, d.x)),
            x2 = min(max(a.x, b.x), max(c.x, d.x));
        ll y1 = max(min(a.y, b.y), min(c.y, d.y)),
            y2 = min(max(a.y, b.y), max(c.y, d.y));
        return (x1 <= x2 && y1 <= y2);
    }
    return false;
}

bool lines_intersection(const Point &a, const Point &b, const
    Point &c,
                        const Point &d) { // line ab, line cd
    Point ab = b - a, cd = d - c, ac = c - a;
    return ((ab % cd) != 0 || (ab % ac == 0));
}

bool line_ray_intersection(const Point &a, const Point &b,
    const Point &c,
                        const Point &d) { // line ab, ray cd
    if (!lines_intersection(a, b, c, d)) {
        return false;
    }
    Point ab = b - a, dp = d + (a - c), ac = c - a;
    if (get_sign(ab % (dp - a)) * get_sign(ab % ac) <= 0) {
        return true;
    }
    return false;
}

Point get_inf(const Point &a, const Point &b) { // returns inf
    point on ray ab
    if (a.x == b.x) {
        if (a.y < b.y) {
            return {a.x, INFLL};
        } else {
            return {a.x, -INFLL};
        }
    }
    if (a.x < b.x) {
        if (a.y < b.y) {
            return {INFLL, INFLL};
        } else if (a.y == b.y) {
            return {INFLL, a.y};
        } else {
            return {INFLL, -INFLL};
        }
    }
    // a.x > b.x
    if (a.y < b.y) {
        return {-INFLL, INFLL};
    } else if (a.y == b.y) {
        return {-INFLL, a.y};
    } else {
        return {-INFLL, -INFLL};
    }
}

bool rays_intersection(const Point &a, const Point &b, const
    Point &c,
                        const Point &d) { // ray ab, ray cd
    if (line_ray_intersection(a, b, c, d) &&
        line_ray_intersection(c, d, a, b)) {
        Point bp = get_inf(a, b), dp = get_inf(c, d);
        ll x1 = max(min(a.x, bp.x), min(c.x, dp.x)),
            x2 = min(max(a.x, bp.x), max(c.x, dp.x));
        ll y1 = max(min(a.y, bp.y), min(c.y, dp.y)),
            y2 = min(max(a.y, bp.y), max(c.y, dp.y));
    }
}

```

```

        return (x1 <= x2 && y1 <= y2);
    }
    return false;
}

bool segment_line_intersection(const Point &a, const Point &b,
    const Point &c,
                        const Point &d) { // segment ab,
                                        line cd
    Point cd = d - c, ca = a - c, cb = b - c;
    if (get_sign(cd % ca) * get_sign(cd % cb) <= 0) {
        return true;
    }
    return false;
}

bool ray_segment_intersection(const Point &a, const Point &b,
    const Point &c,
                        const Point &d) { // ray ab,
                                        segment cd
    if (line_ray_intersection(c, d, a, b) &&
        segment_line_intersection(c, d, a, b)) {
        Point bp = get_inf(a, b);
        ll x1 = max(min(a.x, bp.x), min(c.x, d.x)),
            x2 = min(max(a.x, bp.x), max(c.x, d.x));
        ll y1 = max(min(a.y, bp.y), min(c.y, d.y)),
            y2 = min(max(a.y, bp.y), max(c.y, d.y));
        return (x1 <= x2 && y1 <= y2);
    }
    return false;
}

```

diameter.cpp

Description: Rotating calipers.

Time: $\mathcal{O}(n)$

0adc5e, 18 lines

```

ll diameter_of_polygon(Polygon &poly) {
    int n = sz(poly), x = 1;
    ll ans = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        while (true) {
            Point ac = poly[x] - poly[i], ab = poly[j] - poly[i],
                cd = poly[(x + 1) % n] - poly[x];
            ans = max(ans, ac.len_sq());
            if ((ab % cd) <= 0) {
                break;
            }
            x++;
            x %= n;
        }
    }
    return sqrtl(ans);
}

```

minkowski.cpp

Description: Minkowski sum of two polygons and its applications.

Time: $\mathcal{O}(|A| + |B|)$

db126d, 74 lines

```

Polygon minkowski_sum(Polygon &a, Polygon &b) {
    // a[0], b[0]: y - max, y1 = y2 => x - max. counter clockwise
    int n = sz(a), m = sz(b);
    assert(n >= 3 && m >= 3);
    Point high_a = a[0], high_b = b[0];
    Polygon va(n), vb(m);
    for (int i = 0; i < n; ++i) {
        va[i] = a[(i + 1) % n] - a[i];
    }
    for (int i = 0; i < m; ++i) {

```

```

        vb[i] = b[(i + 1) % m] - b[i];
    }
    // sort(all(va), cmp_vectors);
    // sort(all(vb), cmp_vectors);
    Polygon vc(sz(va) + sz(vb));
    merge(all(va), all(vb), vc.begin(), cmp_vectors);
    Point high_c(high_a.x + high_b.x, high_a.y + high_b.y);
    Polygon c(sz(vc) + 1);
    c[0] = high_c;
    for (int i = 0; i < sz(c) - 1; ++i) {
        c[i + 1] = c[i] + vc[i];
    }
    return c;
}

ld from_polygon_to_polygon(Polygon a, Polygon b) {
    for (auto &i : b) {
        i *= -1;
    }
    int pos = 0;
    for (int i = 1; i < sz(b); ++i) {
        if ((b[i].y > b[pos].y) || (b[i].y == b[pos].y && b[i].x >
            b[pos].x)) {
            pos = i;
        }
    }
    rotate(b.begin(), b.begin() + pos, b.end());
    Polygon c = minkowski_sum(a, b);
    int n = sz(c);
    Point p(0, 0);
    ld ans = 1e20;
    for (int i = 0; i < n - 1; ++i) {
        ans = min(ans, from_point_to_segment(p, c[i], c[i + 1]));
    }
    return ans;
}

ld diameter_of_polygon_minkowski(Polygon &a) {
    Polygon ra = a;
    for (auto &i : ra) {
        i *= -1;
    }
    int pos = 0;
    for (int i = 1; i < sz(a); ++i) {
        if ((a[i].y > a[pos].y) || (a[i].y == a[pos].y && a[i].x >
            a[pos].x)) {
            pos = i;
        }
    }
    rotate(a.begin(), a.begin() + pos, a.end());
    pos = 0;
    for (int i = 1; i < sz(a); ++i) {
        if ((ra[i].y > ra[pos].y) ||
            (ra[i].y == ra[pos].y && ra[i].x > ra[pos].x)) {
            pos = i;
        }
    }
    rotate(ra.begin(), ra.begin() + pos, ra.end());
    Polygon c = minkowski_sum(a, ra);
    int n = sz(c);
    ll ans = 0;
    for (int i = 0; i < n; ++i) {
        ans = max(ans, c[i].len_sq());
    }
    return sqrtl(ans);
}

```

tangents.cpp
Description: Returns id of tangent point: (left tangent, right tangent) (maybe (right, left), but i believe no).

```
e40872, 26 lines
pii tangent_from_point(const Point &p, const Polygon &poly) {
    int n = sz(poly);
    int i_min = 0;
    int i_max = 0;
    for (int k = 17; k >= 0; --k) {
        {
            int l = (i_min + (1 << k)) % n;
            int r = ((i_min - (1 << k)) % n + n) % n;
            i_min = min({l, r, i_min}, [&poly, &p](int i, int j) {
                return ((poly[i] - p) % (poly[j] - p)) < 0 ||
                    (((poly[i] - p) % (poly[j] - p)) == 0 &&
                     (poly[i] - p).len_sq() < (poly[j] - p).len_sq())
                    );
            });
        }
        {
            int l = (i_max + (1 << k)) % n;
            int r = ((i_max - (1 << k)) % n + n) % n;
            i_max = max({l, r, i_max}, [&poly, &p](int i, int j) {
                return ((poly[i] - p) % (poly[j] - p)) < 0 ||
                    (((poly[i] - p) % (poly[j] - p)) == 0 &&
                     (poly[i] - p).len_sq() > (poly[j] - p).len_sq())
                    );
            });
        }
    }
    return {i_min, i_max};
}
```

circle.cpp
Description: Class to handle circle in the plane. T can be e.g. double or long long. (Avoid int.)

```
6e8a38, 53 lines
template <typename T> struct circle_t {
    point_t<T> c;
    T r;

    circle_t() {}

    circle_t(point_t<T> _c, T _r) : c(_c), r(_r) {}

    int point_in(const point_t<T> &a) {
        if (is_same<T, ll>()) {
            ll dist = (a - c).len_sq();
            if (dist > r * r) {
                return 0;
            } else if (dist < r * r) {
                return 1;
            } else {
                return 2;
            }
        } else {
            ld dist = (a - c).len();
            if (dist > r + EPS) {
                return 0;
            } else if (dist + EPS < r) {
                return 1;
            } else {
                return 2;
            }
        }
    }

    vector<PointLD> tangent_from_point(const point_t<T> &a) {
        PointLD p(a.x, a.y);
        PointLD vec(c.x - p.x, c.y - p.y);
```

```

        ld dist = vec.len();
        if (dist + EPS < r) {
            return {};
        }
        if (abs(r - dist) < EPS) {
            return {p};
        }
        vec.x /= dist;
        vec.y /= dist;
        ld k = sqrt(1 - dist * dist - r * r);
        ld alpha = atan2(r, k);
        PointLD t1 = p + rotate(vec, alpha) * k, t2 = p + rotate(
            vec, -alpha) * k;
        return {t1, t2};
    }
};
```

```

template <typename T> istream &operator>>(istream &is, circle_t
    <T> &c) {
    is >> c.c >> c.r;
    return is;
}
```

halfplanes.cpp
Description: half-plane intersection area
Time: $\mathcal{O}(N \log N)$

```
9dca58, 121 lines
using Half = array<ld, 3>; // half-plane, ax + by >= c
using vH = vector<Half>;

PointLD hp_point(const Half &h) {
    return {h[0], h[1]};
} // direction of half-plane
PointLD isect(const Half &h0,
    const Half &h1) { // Cramer's rule to intersect
    half-planes

    array<ld, 3> vals{};
    for (int i = -1; i <= 1; ++i) {
        int x = (i == 0 ? 2 : 0), y = (i == 1 ? 2 : 1);
        vals[1 + i] = h0[x] * h1[y] - h0[y] * h1[x];
    }
    assert(fabs1(vals[0]) > EPS);
    return {vals[1] / vals[0], vals[2] / vals[0]};
}

ld eval(const Half &h, ld x) { // evaluate half-plane at x-
    coordinate
    assert(fabs1(h[1]) > EPS);
    return (h[2] - h[0] * x) / h[1];
}

ld x_isect(const Half &h0, const Half &h1) {
    return isect(h0, h1).x;
} // x-coordinate of intersection

vH construct_lower(PointLD x,
    vH planes) { // similar to convex hull (by
    duality)

    sort(all(planes), [](const Half &a, const Half &b) {
        return hp_point(a) % hp_point(b) > EPS;
    });
    vH res{{1, 0, x.x}}; // >= x.f
    planes.push_back({-1, 0, -x.y}); // <= x.s
    auto lst_x = [&](Half a, Half b) {
        if (fabs1(hp_point(a) % hp_point(b)) <=
            EPS) { // parallel half-planes, remove lower one
            return a[2] / a[1] <= b[2] / b[1] ? x.x : x.y;
        }
        return x_isect(a, b);
    };
```

```

};
for (auto t : planes) {
    while (sz(res) > 1 &&
        lst_x(res.back(), t) <= lst_x(res[sz(res) - 2], res.
            back())) {
        res.pop_back();
    }
    res.push_back(t);
}
return res;
}

ld isect_area(vH planes) {
    const ld BIG = 1e9;
    PointLD x{-BIG, BIG};
    planes.push_back({0, 1, -BIG}); // y >= -BIG
    planes.push_back({0, -1, -BIG}); // -y >= -BIG
    vH upper, lower;
    for (auto &t : planes) {
        if (fabs1(t[1]) <= EPS) { // vertical line
            ld quo = t[2] / t[0];
            if (t[0] > 0) {
                if (quo > x.x)
                    x.x = quo;
            } else { // -x >=
                if (quo < x.y)
                    x.y = quo;
            }
        } else if (t[1] > 0) {
            lower.push_back(t);
        } else {
            upper.push_back(t);
        }
    }
    if (x.x >= x.y)
        return 0;
    lower = construct_lower(x, lower);
    for (auto &t : upper) {
        t[0] *= -1;
        t[1] *= -1;
    }
    upper = construct_lower({-x.y, -x.x}, upper);
    for (auto &t : upper) {
        t[0] *= -1;
        t[1] *= -1;
    }
    reverse(all(upper));
    int iu = 1, il = 1;
    ld lst = x.x, lst_dif = eval(upper[1], lst) - eval(lower[1],
        lst);
    ld ans = 0;
    while (iu < sz(upper) - 1 &&
        il < sz(lower) -
            1) { // sweep vertical line through lower and
                upper hulls
        ld nex_upper = x_isect(upper[iu], upper[iu + 1]);
        ld nex_lower = x_isect(lower[il], lower[il + 1]);
        ld nex = min(nex_upper, nex_lower);
        ld nex_dif = eval(upper[iu], nex) - eval(lower[il], nex);
        auto avg_val = [](ld a, ld b) -> ld {
            if (a > b)
                swap(a, b);
            if (b <= 0)
                return 0;
            if (a >= 0)
                return (a + b) / 2;
            return b / (b - a) * b / 2;
        };
        ans += (nex - lst) * avg_val(lst_dif, nex_dif);
```

```
    assert(x.x <= nex && nex <= x.y);
    lst = nex, lst_dif = nex_dif;
    iu += fabs1(lst - nex_upper) <= EPS;
    il += fabs1(lst - nex_lower) <= EPS;
}
return ans;
}

Half plane_right(PointLD a, PointLD b) { // half-plane to right
of a -> b
    return {b.y - a.y, a.x - b.x, (b.y - a.y) * a.x + (a.x - b.x)
        * a.y};
}

Half plane_through(PointLD p,
    PointLD dir) { // half-plane through p in
direction dir
    return {dir.x, dir.y, p * dir};
}
```

The Equation List (8)

8.1 Combinatorics

8.1.1 General

- 1. $\sum_{0 \leq k \leq n} \binom{n-k}{k} = Fib_{n+1}$
- 2. $\binom{n}{k} = \binom{n}{n-k}$
- 3. $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- 4. $k \binom{n}{k} = n \binom{n-1}{k-1}$
- 5. $\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1}$
- 6. $\sum_{i=0}^n \binom{n}{i} = 2^n$
- 7. $\sum_{i \geq 0} \binom{n}{2i} = 2^{n-1}$
- 8. $\sum_{i \geq 0} \binom{n}{2i+1} = 2^{n-1}$
- 9. $\sum_{i=0}^k (-1)^i \binom{n}{i} = (-1)^k \binom{n-1}{k}$
- 10. $\sum_{i=0}^k \binom{n+i}{i} = \sum_{i=0}^k \binom{n+i}{n} = \binom{n+k+1}{k}$

- 11. $1 \binom{n}{1} + 2 \binom{n}{2} + 3 \binom{n}{3} + \dots + n \binom{n}{n} = n 2^{n-1}$
- 12. $1^2 \binom{n}{1} + 2^2 \binom{n}{2} + 3^2 \binom{n}{3} + \dots + n^2 \binom{n}{n} = (n + n^2) 2^{n-2}$
- 13. **Vandermonde’s Identify:**
 $\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$
- 14. **Hockey-Stick Identify:**
 $n, r \in N, n > r, \sum_{i=r}^n \binom{i}{r} = \binom{n+1}{r+1}$
- 15. $\sum_{i=0}^k \binom{k}{i}^2 = \binom{2k}{k}$
- 16. $\sum_{k=0}^n \binom{n}{k} \binom{n}{n-k} = \binom{2n}{n}$
- 17. $\sum_{k=q}^n \binom{n}{k} \binom{k}{q} = 2^{n-q} \binom{n}{q}$
- 18. $\sum_{i=0}^n k^i \binom{n}{i} = (k+1)^n$
- 19. $\sum_{i=0}^n 2 \binom{n}{i} = 2^{2n-1} + \frac{1}{2} \binom{2n}{n}$
- 20. $\sum_{i=1}^n \binom{n}{i} \binom{n-1}{i-1} = \binom{2n-1}{n-1}$
- 21. $\sum_{i=0}^n 2 \binom{n}{i}^2 = \frac{1}{2} \left(4 \binom{n}{2} n + \binom{2n}{n}^2 \right)$
- 22. **Highest Power of 2 that divides $^{2n}C_n$:** Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x . Let it form a sequence. The n -th value in the sequence (starting from $n = 0$) gives the highest power of 2 that divides $^{2n}C_n$.
- 23. **Pascal Triangle**
 - (a) In a row p where p is a prime number, all the terms in that row except the 1s are multiples of p .
 - (b) Parity: To count odd terms in row n , convert n to binary. Let x be the number of 1s in the binary representation. Then the number of odd terms will be 2^x .
 - (c) Every entry in row $2^n - 1, n \geq 0$, is odd.

- 24. An integer $n \geq 2$ is prime if and only if all the intermediate binomial coefficients $\binom{n}{1}, \binom{n}{2}, \dots, \binom{n}{n-1}$ are divisible by n .
- 25. **Kummer’s Theorem:** For given integers $n \geq m \geq 0$ and a prime number p , the largest power of p dividing $\binom{n}{m}$ is equal to the number of carries when m is added to $n-m$ in base p . For implementation take inspiration from lucas theorem.
- 26. Number of different binary sequences of length n such that no two 0’s are adjacent= Fib_{n+1}
- 27. **Combination with repetition:** Let’s say we choose k elements from an n -element set, the order doesn’t matter and each element can be chosen more than once. In that case, the number of different combinations is: $\binom{n+k-1}{k}$
- 28. Number of ways to divide n persons in $\frac{n}{k}$ equal groups i.e. each having size k is

$$\frac{n!}{k!^{\frac{n}{k}} (\frac{n}{k})!} = \prod_{n \geq k} \binom{n-1}{k-1}$$

- 29. The number non-negative solution of the equation:
 $x_1 + x_2 + x_3 + \dots + x_k = n$ is $\binom{n+k-1}{n}$
- 30. Number of ways to choose n ids from 1 to b such that every id has distance at least k = $\binom{b-(n-1)(k-1)}{n}$
- 31. $\sum_{i=1,3,5,\dots}^{i \leq n} \binom{n}{i} a^{n-i} b^i = \frac{1}{2} ((a+b)^n - (a-b)^n)$
- 32. $\sum_{i=0}^n \frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{k}}$
- 33. **Derangement:** a permutation of the elements of a set, such that no element appears in its original position. Let $d(n)$ be the number of derangements of the identity permutation fo size n .

$$d(n) = (n-1) \cdot (d(n-1) + d(n-2)) \text{ where } d(0) = 1, d(1) = 0$$

34. **Involutions:** permutations such that $p^2 = \text{identity}$ permutation. $a_0 = a_1 = 1$ and $a_n = a_{n-1} + (n-1)a_{n-2}$ for $n > 1$.
35. Let $T(n, k)$ be the number of permutations of size n for which all cycles have length $\leq k$.

$$T(n, k) = \begin{cases} n! , n \leq k \\ n \cdot T(n-1, k) - F(n-1, k) \cdot T(n-k-1, k) \\ , n > k \end{cases}$$

Here $F(n, k) = n \cdot (n-1) \cdot \dots \cdot (n-k+1)$

36. Lucas Theorem

- (a) If p is prime, then $\left(\frac{p^a}{k}\right) \equiv 0 \pmod{p}$
- (b) For non-negative integers m and n and a prime p , the following congruence relation holds:

$$\left(\frac{m}{n}\right) \equiv \prod_{i=0}^k \left(\frac{m_i}{n_i}\right) \pmod{p},$$

where, $m = m_k p^k + m_{k-1} p^{k-1} + \dots + m_1 p + m_0$, and $n = n_k p^k + n_{k-1} p^{k-1} + \dots + n_1 p + n_0$ are the base p expansions of m and n respectively. This uses the convention that $\left(\frac{m}{n}\right) = 0$, when $m < n$.

$$\begin{aligned} 37. \sum_{i=0}^n \binom{n}{i} \cdot i^k &= \sum_{i=0}^n \binom{n}{i} \cdot \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot i^j \\ &= \sum_{i=0}^n \binom{n}{i} \cdot \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot j! \binom{n}{i} \\ &= \sum_{i=0}^n \frac{n!}{(n-i)!} \cdot \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(i-j)!} \\ &= \sum_{i=0}^n \sum_{j=0}^k \frac{n!}{(n-i)!} \cdot \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(i-j)!} \\ &= n! \sum_{i=0}^n \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(n-i)!} \cdot \frac{1}{(i-j)!} \\ &= n! \sum_{i=0}^n \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \binom{n-j}{n-i} \cdot \frac{1}{(n-j)!} \\ &= n! \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot \frac{1}{(n-j)!} \sum_{i=0}^n \binom{n-j}{n-i} = \sum_{j=0}^k \left\{ \begin{matrix} k \\ j \end{matrix} \right\} \cdot n^j \cdot 2^{n-j} \end{aligned}$$

Here $n^j = P(n, j) = \frac{n!}{(n-j)!}$ and $\left\{ \begin{matrix} k \\ j \end{matrix} \right\}$ is stirling number of the second kind.

So, instead of $O(n)$, now you can calculate the original equation in $O(k^2)$ or even in $O(k \log^2 n)$ using NTT.

38. $\sum_{i=0}^{n-1} \binom{n}{i} x^i = x^j (1-x)^{-j-1} \left(1 - x^n \sum_{i=0}^j \binom{n}{i} x^{j-i} (1-x)^i \right)$
39. $x_0, x_1, x_2, x_3, \dots, x_n$ $x_0 + x_1, x_1 + x_2, x_2 + x_3, \dots, x_n$... If we continuously do this n times then the polynomial of the first column of the n -th row will be

$$p(n) = \sum_{k=0}^n \binom{n}{k} \cdot x(k)$$

40. If $P(n) = \sum_{k=0}^n \binom{n}{k} \cdot Q(k)$, then,

$$Q(n) = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} \cdot P(k)$$

41. If $P(n) = \sum_{k=0}^n (-1)^k \binom{n}{k} \cdot Q(k)$, then,

$$Q(n) = \sum_{k=0}^n (-1)^k \binom{n}{k} \cdot P(k)$$

8.1.2 Catalan Numbers

- $C_n = \frac{1}{n+1} \binom{2n}{n}$
- $C_0 = 1, C_1 = 1$ and $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$
- Number of correct bracket sequence consisting of n opening and n closing brackets.
- The number of ways to completely parenthesize $n+1$ factors.
- The number of triangulations of a convex polygon with $n+2$ sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
- The number of ways to connect the $2n$ points on a circle to form n disjoint i.e. non-intersecting chords.
- The number of monotonic lattice paths from point $(0, 0)$ to point (n, n) in a square lattice of size $n \times n$, which do not pass above the main diagonal (i.e. connecting $(0, 0)$ to (n, n)).

- The number of rooted full binary trees with $n+1$ leaves (vertices are not numbered). A rooted binary tree is full if every vertex has either two children or no children.
- Number of permutations of $\{1, \dots, n\}$ that avoid the pattern 123 (or any of the other patterns of length 3); that is, the number of permutations with no three-term increasing sub-sequence. For $n = 3$, these permutations are 132, 213, 231, 312 and 321. For $n = 4$, they are 1432, 2143, 2413, 2431, 3142, 3214, 3241, 3412, 3421, 4132, 4213, 4231, 4312 and 4321.
- Balanced Parentheses count with prefix:** The count of balanced parentheses sequences consisting of $n + k$ pairs of parentheses where the first k symbols are open brackets. Let the number be $C_n^{(k)}$, then

$$C_n^{(k)} = \frac{k+1}{n+k+1} \binom{2n+k}{n}$$

8.1.3 Narayana numbers

- $N(n, k) = \frac{1}{n} \binom{n}{k} \binom{n}{k-1}$
- The number of expressions containing n pairs of parentheses, which are correctly matched and which contain k distinct nestings. For instance, $N(4, 2) = 6$ as with four pairs of parentheses six sequences can be created which each contain two times the sub-pattern $'()'$.

8.1.4 Stirling numbers of the first kind

- The Stirling numbers of the first kind count permutations according to their number of cycles (counting fixed points as cycles of length one).
- $S(n, k)$ counts the number of permutations of n elements with k disjoint cycles.
- $S(n, k) = (n-1) \cdot S(n-1, k) + S(n-1, k-1)$,
where, $S(0, 0) = 1, S(n, 0) = S(0, n) = 0$
- $\sum_{k=0}^n S(n, k) = n!$
- The unsigned Stirling numbers may also be defined algebraically, as the coefficient of the rising factorial:

$$x^{\bar{n}} = x(x+1) \dots (x+n-1) = \sum_{k=0}^n S(n, k) x^k$$

- Lets $[n, k]$ be the stirling number of the first kind, then

$$\left[\begin{matrix} n \\ k \end{matrix} \right] = \sum_{0 \leq i_1 < i_2 < \dots < i_k \leq n} i_1 i_2 \dots i_k.$$

8.1.5 Stirling numbers of the second kind

1. Stirling number of the second kind is the number of ways to partition a set of n objects into k non-empty subsets.
2. $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$,

$$\text{where } S(0, 0) = 1, S(n, 0) = S(0, n) = 0$$

3. $S(n, 2) = 2^{n-1} - 1$
4. $S(n, k) \cdot k!$ = number of ways to color n nodes using colors from 1 to k such that each color is used at least once.
5. An r -associated Stirling number of the second kind is the number of ways to partition a set of n objects into k subsets, with each subset containing at least r elements. It is denoted by $S_r(n, k)$ and obeys the recurrence relation.

$$S_r(n+1, k) = k S_r(n, k) + \binom{n}{r-1} S_r(n-r+1, k-1)$$
6. Denote the n objects to partition by the integers $1, 2, \dots, n$. Define the reduced Stirling numbers of the second kind, denoted $S^d(n, k)$, to be the number of ways to partition the integers $1, 2, \dots, n$ into k nonempty subsets such that all elements in each subset have pairwise distance at least d . That is, for any integers i and j in a given subset, it is required that $|i - j| \geq d$. It has been shown that these numbers satisfy,

$$S^d(n, k) = S(n-d+1, k-d+1), n \geq k \geq d$$

8.1.6 Bell number

1. Counts the number of partitions of a set.
2. $B_{n+1} = \sum_{k=0}^n \binom{n}{k} \cdot B_k$
3. $B_n = \sum_{k=0}^n S(n, k)$, where $S(n, k)$ is Stirling number of second kind.

8.2 Math

8.2.1 General

1. $ab \bmod ac = a(b \bmod c)$
2. $\sum_{i=0}^n i \cdot i! = (n+1)! - 1$.
3. $a^k - b^k = (a-b) \cdot (a^{k-1}b^0 + a^{k-2}b^1 + \dots + a^0b^{k-1})$

4. $\min(a+b, c) = a + \min(b, c-a)$
5. $\|a-b\| + \|b-c\| + \|c-a\| = 2(\max(a, b, c) - \min(a, b, c))$
6. $a \cdot b \leq c \rightarrow a \leq \left\lfloor \frac{c}{b} \right\rfloor$ is correct
7. $a \cdot b < c \rightarrow a < \left\lfloor \frac{c}{b} \right\rfloor$ is incorrect
8. $a \cdot b \geq c \rightarrow a \geq \left\lfloor \frac{c}{b} \right\rfloor$ is correct
9. $a \cdot b > c \rightarrow a > \left\lfloor \frac{c}{b} \right\rfloor$ is correct
10. For positive integer n , and arbitrary real numbers m, x ,

$$\left\lfloor \frac{\lfloor x/m \rfloor}{n} \right\rfloor = \left\lfloor \frac{x}{mn} \right\rfloor$$

$$\left\lceil \frac{\lceil x/m \rceil}{n} \right\rceil = \left\lceil \frac{x}{mn} \right\rceil$$
11. Lagrange's identity:

$$\left(\sum_{k=1}^n a_k^2 \right) \left(\sum_{k=1}^n b_k^2 \right) - \left(\sum_{k=1}^n a_k b_k \right)^2$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^n (a_i b_j - a_j b_i)^2 = \frac{1}{2} \sum_{i=1}^n \sum_{j=1, j \neq i}^n (a_i b_j - a_j b_i)^2$$
12. $\sum_{i=1}^n i a^i = \frac{a(na^{n+1} - (n+1)a^n + 1)}{(a-1)^2}$
13. **Vieta's formulas:** Any general polynomial of degree n

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

(with the coefficients being real or complex numbers and $a_n \neq 0$) is known by the fundamental theorem of algebra to have n (not necessarily distinct) complex roots $r_1, r_2, \dots, r_n$.

$$\begin{cases} r_1 + r_2 + \dots + r_{n-1} + r_n = -\frac{a_{n-1}}{a_n} \\ (r_1 r_2 + r_1 r_3 + \dots + r_1 r_n) + (r_2 r_3 + r_2 r_4 + \dots + r_2 r_n) + \\ + \dots + r_{n-1} r_n = \frac{a_{n-2}}{a_n} \\ \vdots \\ r_1 r_2 \dots r_n = (-1)^n \frac{a_0}{a_n} \end{cases}$$

Vieta's formulas can equivalently be written as

$$\sum_{1 \leq i_1 < i_2 < \dots < i_k \leq n} \left(\prod_{j=1}^k r_{i_j} \right) = (-1)^k \frac{a_{n-k}}{a_n},$$

14. We are given n numbers $a_1, a_2, \dots, a_n$ and our task is to find a value x that minimizes the sum,

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|$$

optimal x = median of the array. if n is even x = [left median, right median] i.e. every number in this range will work.

For minimizing

$$(a_1 - x)^2 + (a_2 - x)^2 + \dots + (a_n - x)^2$$

$$\text{optimal } x = \frac{(a_1 + a_2 + \dots + a_n)}{n}$$

15. Given an array a of n non-negative integers. The task is to find the sum of the product of elements of all the possible subsets. It is equal to the product of $(a_i + 1)$ for all a_i
16. **Pentagonal number theorem:** In mathematics, the pentagonal number theorem states that

$$\prod_{n=1}^{\infty} (1 - x^n) = \prod_{k=-\infty}^{\infty} (-1)^k x^{\frac{k(3k-1)}{2}} = 1 + \prod_{k=1}^{\infty} (-1)^k \left(x^{\frac{k(3k+1)}{2}} + x^{\frac{k(3k-1)}{2}} \right)$$

In other words,

$$(1-x)(1-x^2)(1-x^3) \dots = 1 - x - x^2 + x^5 + x^7 - x^{12} - x^{15} + x^{22} + x^{26} - \dots$$

The exponents $1, 2, 5, 7, 12, \dots$ on the right hand side are given by the formula $g_k = \frac{k(3k-1)}{2}$ for $k = 1, -1, 2, -2, 3, \dots$ and are called (generalized) pentagonal numbers.

It is useful to find the partition number in $O(n\sqrt{n})$

8.2.2 Fibonacci Number

- 1. $F_0 = 0, F_1 = 1$ and $F_n = F_{n-1} + F_{n-2}$
- 2. $F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$
- 3. $F_n = \frac{1}{\sqrt{5}}(\frac{1+\sqrt{5}}{2})^n - \frac{1}{\sqrt{5}}(\frac{1-\sqrt{5}}{2})^n$
- 4. $\sum_{i=1}^n F_i = F_{n+2} - 1$
- 5. $\sum_{i=0}^{n-1} F_{2i+1} = F_{2n}$
- 6. $\sum_{i=1}^n F_{2i} = F_{2n+1} - 1$
- 7. $\sum_{i=1}^n F_i^2 = F_n F_{n+1}$
- 8. $\frac{F_m F_{n+1} - F_{m-1} F_n}{F_{2n}} = \frac{(-1)^n F_{m-n}}{F_{n+1}^2 - F_{n-1}^2} = F_n(F_{n+1} + F_{n-1})$
- 9. $F_m F_n + F_{m-1} F_{n-1} = F_{m+n-1}$ $F_m F_{n+1} + F_{m-1} F_n = F_{m+n}$
- 10. A number is Fibonacci if and only if one or both of $(5 \cdot n^2 + 4)$ or $(5 \cdot n^2 - 4)$ is a perfect square
- 11. Every third number of the sequence is even and more generally, every k^{th} number of the sequence is a multiple of F_k
- 12. $gcd(F_m, F_n) = F_{gcd(m,n)}$
- 13. Any three consecutive Fibonacci numbers are pairwise coprime, which means that, for every n, $gcd(F_n, F_{n+1}) = gcd(F_n, F_{n+2}), gcd(F_{n+1}, F_{n+2}) = 1$
- 14. If the members of the Fibonacci sequence are taken *mod* n, the resulting sequence is periodic with period at most 6n.

8.2.3 Pythagorean Triples

- 1. A Pythagorean triple consists of three positive integers a, b , and C , such that $a^2 + b^2 = c^2$. Such a triple is commonly written (a, b, c)
- 2. Euclid’s formula is a fundamental formula for generating Pythagorean triples given an arbitrary pair of integers m and n with $m > n > 0$. The formula states that the integers

$a = m^2 - n^2, b = 2mn, c = m^2 + n^2$

form a Pythagorean triple. The triple generated by Euclid’s formula is primitive if and only if m and n are coprime and not both odd. When both m and n are odd, then a, b, and c will be even, and the triple will not be primitive; however, dividing a, b, and c by 2 will yield a primitive triple when m and n are coprime and both odd.

- 3. The following will generate all Pythagorean triples uniquely:

$a = k \cdot (m^2 - n^2), b = k \cdot (2mn), c = k \cdot (m^2 + n^2)$

where m, n, and k are positive integers with $m > n$, and with m and n coprime and not both odd.

- 4. **Theorem:** The number of Pythagorean triples a,b,n with $max\{a, b, n\} = n$ is given by

$\frac{1}{2} \left(\prod_{p^\alpha || n} (2\alpha + 1) - 1 \right)$

where the product is over all prime divisors p of the form $4k + 1$. The notation $p^\alpha || n$ stands for the highest exponent α for which p^α divides n **Example:** For $n = 2 \cdot 3^2 \cdot 5^3 \cdot 7^4 \cdot 11^5 \cdot 13^6$, the number of Pythagorean triples with hypotenuse n is $\frac{1}{2} (7 \cdot 13 - 1) = 45$. To obtain a formula for the number of Pythagorean triples with hypotenuse less than a specific positive integer N, we may add the numbers corresponding to each $n < N$ given by the Theorem. There is no simple way to compute this as a function of N.

8.2.4 Sum of Squares Function

- 1. The function is defined as $r_k(n) = |\{(a_1, a_2, \dots, a_k) \in \mathbf{Z}^k : n = a_1^2 + a_2^2 + \dots + a_k^2\}|$
- 2. The number of ways to write a natural number as sum of two squares is given by $r_2(n)$. It is given explicitly by $r_2(n) = 4(d_1(n) - d_3(n))$ where d1(n) is the number of divisors of n which are congruent with 1 modulo 4 and d3(n) is the number of divisors of n which are congruent with 3 modulo 4. $\sum_{d|n} \chi(d)$ The prime factorization $n = 2^g p_1^{f_1} p_2^{f_2} \dots q_1^{h_1} q_2^{h_2} \dots$, where p_i are the prime factors of the form $p_i \equiv 1 \pmod{4}$, and q_i are the prime factors of the form $q_i \equiv 3 \pmod{4}$ gives another formula $r_2(n) = 4(f_1 + 1)(f_2 + 1) \dots$, if all exponents $h_1, h_2, \dots$ are even. If one or more h_i are odd, then $r_2(n) = 0$.

- 3. The number of ways to represent n as the sum of four squares is eight times the sum of all its divisors which are not divisible by 4, i.e. $r_4(n) = 8 \sum_{d|n; 4 \nmid d} d$
 $r_8(n) = 16 \sum_{d|n} (-1)^{n+d} d^3$

8.3 Number Theory

8.3.1 General

- 1. for $i > j, gcd(i, j) = gcd(i - j, j) \leq (i - j)$
- 2. $\sum_{x=1}^n [d||x^k] = \left\lfloor \frac{n}{\prod_{i=0}^n p_i^{\left\lceil \frac{e_i}{k} \right\rceil}} \right\rfloor$,
where $d = \prod_{i=0}^n p_i^{e_i}$. Here, $[a||b]$ means if a divides b then it is 1, otherwise it is 0.
- 3. The number of lattice points on segment (x_1, y_1) to (x_2, y_2) is $gcd(abs(x_1 - x_2), abs(y_1 - y_2)) + 1$
- 4. $(n - 1)! \pmod n = n - 1$ if n is prime, 2 if $n = 4$, 0 otherwise.
- 5. A number has odd number of divisors if it is perfect square
- 6. The sum of all divisors of a natural number n is odd if and only if $n = 2^r \cdot k^2$ where r is non-negative and k is positive integer.
- 7. Let a and b be coprime positive integers, and find integers a' and b' such that $aa' \equiv 1 \pmod b$ and $bb' \equiv 1 \pmod a$. Then the number of representations of a positive integers n as a non negative linear combination of a and b is

$\frac{n}{ab} - \left\{ \frac{bn}{a} \right\} - \left\{ \frac{an}{b} \right\} + 1$

Here, $\{x\}$ denotes the fractional part of x .

- 8. $\sum_{i=1}^a \sum_{j=1}^b \sum_{k=1}^c d(i \cdot j \cdot k) = \sum_{gcd(i,j)=gcd(j,k)=gcd(k,i)=1} \left\lfloor \frac{a}{i} \right\rfloor \left\lfloor \frac{b}{j} \right\rfloor \left\lfloor \frac{c}{k} \right\rfloor$
Here, $d(x)$ = number of divisors of x .
- 9. **Gauss’s generalization of Wilson’s theorem:**, Gauss proved that,

$\prod_{\substack{k=1 \\ gcd(k,m)=1}}^m k \equiv \begin{cases} -1 \pmod m & \text{if } m = 4, p^\alpha, 2p^\alpha \\ 1 \pmod m & \text{otherwise} \end{cases}$

where p represents an odd prime and α a positive integer. The values of m for which the product is -1 are precisely the ones where there is a primitive root modulo m .

8.3.2 Divisor Function

1. $\sigma_x(n) = \sum_{d|n} d^x$
2. It is multiplicative i.e if $\gcd(a, b) = 1 \rightarrow \sigma_x(ab) = \sigma_x(a)\sigma_x(b)$.
- 3.

$$\sigma_x(n) = \prod_{i=1}^{\tau} \frac{p_i^{(a_i+1)x} - 1}{p_i^x - 1}$$

4. **Divisor Summatory Function** 1. Let $\sigma_0(k)$ be the number of divisors of k . 2. $D(x) = \sum_{n \leq x} \sigma_0(n)$ 3.

$$D(x) = \sum_{k=1}^x \lfloor \frac{x}{k} \rfloor = 2 \sum_{k=1}^u \lfloor \frac{x}{k} \rfloor - u^2, \text{ where } u = \sqrt{x}$$

$D(n)$ = Number of increasing arithmetic progressions where $n+1$ is the second or later term. (i.e. The last term, starting term can be any positive integer $\leq n$. For example, $D(3) = 5$ and there are 5 such arithmetic progressions: $(1, 2, 3, 4)$; $(2, 3, 4)$; $(1, 4)$; $(2, 4)$; $(3, 4)$.

5. Let $\sigma_1(k)$ be the sum of divisors of k . Then,

$$\sum_{k=1}^n \sigma_1(k) = \sum_{k=1}^n k \left\lfloor \frac{n}{k} \right\rfloor$$

6. $\prod_{d|n} d = n^{\frac{\sigma_0-1}{2}}$ if n is not a perfect square, and $= \sqrt{n} \cdot n^{\frac{\sigma_0-1}{2}}$ if n is a perfect square.

8.3.3 Euler's Totient function

1. The function is multiplicative. This means that if $\gcd(m, n) = 1$, $\phi(m \cdot n) = \phi(m) \cdot \phi(n)$.
2. $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$
3. If p is prime and $k \geq 1$, then, $\phi(p^k) = p^{k-1}(p-1) = p^k(1 - \frac{1}{p})$
4. $J_k(n)$, the Jordan totient function, is the number of k -tuples of positive integers all less than or equal to n that form a coprime $(k+1)$ -tuple together with n . It is a generalization of Euler's totient, $\phi(n) = J_1(n)$.

$$J_k(n) = n^k \prod_{p|n} (1 - \frac{1}{p^k})$$

$$5. \sum_{d|n} J_k(d) = n^k$$

$$6. \sum_{d|n} \phi(d) = n$$

$$7. \phi(n) = \sum_{d|n} \mu(d) \cdot \frac{n}{d} = n \sum_{d|n} \frac{\mu(d)}{d}$$

$$8. \phi(n) = \sum_{d|n} d \cdot \mu(\frac{n}{d})$$

$$9. a|b \rightarrow \varphi(a)|\varphi(b)$$

$$10. n|\varphi(a^n - 1) \text{ for } a, n > 1$$

$$11. \varphi(mn) = \varphi(m)\varphi(n) \cdot \frac{d}{\varphi(d)} \text{ where } d = \gcd(m, n) \text{ Note the special cases}$$

$$\varphi(2m) = \begin{cases} 2\varphi(m) & ; \text{if } m \text{ is even} \\ \varphi(m) & ; \text{if } m \text{ is odd} \end{cases}$$

$$\varphi(n^m) = n^{m-1}\varphi(n)$$

$$12. \varphi(\text{lcm}(m, n)) \cdot \varphi(\gcd(m, n)) = \varphi(m) \cdot \varphi(n) \text{ Compare this to the formula } \text{lcm}(m, n) \cdot \gcd(m, n) = m \cdot n$$

$$13. \varphi(n) \text{ is even for } n \geq 3. \text{ Moreover, if } n \text{ has } r \text{ distinct odd prime factors, } 2^r | \varphi(n)$$

$$14. \sum_{d|n} \frac{\mu^2(d)}{\varphi(d)} = \frac{n}{\varphi(n)}$$

$$15. \sum_{1 \leq k \leq n, \gcd(k, n)=1} k = \frac{1}{2} n \varphi(n) \text{ for } n > 1$$

$$16. \frac{\varphi(n)}{n} = \frac{\varphi(\text{rad}(n))}{\text{rad}(n)} \text{ where } \text{rad}(n) = \prod_{p|n, p \text{ prime}} p$$

$$17. \phi(m) \geq \log_2 m$$

$$18. \phi(\phi(m)) \leq \frac{m}{2}$$

$$19. \text{When } x \geq \log_2 m, \text{ then}$$

$$n^x \mod m = n^{\phi(m)+x \mod \phi(m)} \mod m$$

$$20. \sum_{1 \leq k \leq n, \gcd(k, n)=1} \gcd(k-1, n) = \varphi(n)d(n) \text{ where } d(n) \text{ is number of divisors. Same equation for } \gcd(a \cdot k - 1, n) \text{ where } a \text{ and } n \text{ are coprime.}$$

$$21. \text{For every } n \text{ there is at least one other integer } m \neq n \text{ such that } \varphi(m) = \varphi(n).$$

$$22. \sum_{i=1}^n \varphi(i) \cdot \lfloor \frac{n}{i} \rfloor = \frac{n \cdot (n+1)}{2}$$

$$23. \sum_{i=1, i \% 2 \neq 0}^n \varphi(i) \cdot \lfloor \frac{n}{i} \rfloor = \sum_{k \geq 1} \lfloor \frac{n}{2^k} \rfloor^2. \text{ Note that } \lfloor \cdot \rfloor \text{ is used here to denote round operator not floor or ceil}$$

$$24.$$

$$\sum_{i=1}^n \sum_{j=1}^n ij [\gcd(i, j) = 1] = \sum_{i=1}^n \varphi(i) i^2$$

$$25. \text{Average of coprimes of } n \text{ which are less than } n \text{ is } \frac{n}{2}.$$

8.3.4 Mobius Function and Inversion

1. For any positive integer n , define $\mu(n)$ as the sum of the primitive n^{th} roots of unity. It has values in $\{-1, 0, 1\}$ depending on the factorization of n into prime factors: 1. $\mu(n) = 1$ if n is a square-free positive integer with an even number of prime factors. 2. $\mu(n) = -1$ if n is a square-free positive integer with an odd number of prime factors. 3. $\mu(n) = 0$ if n has a squared prime factor.

$$2. \text{It is a multiplicative function.}$$

$$3.$$

$$\sum_{d|n} \mu(d) = \begin{cases} 1 & ; n = 1 \\ 0 & ; n > 0 \end{cases}$$

$$4. \sum_{n=1}^N \mu^2(n) = \sum_{n=1}^{\sqrt{N}} \mu(k) \cdot \left\lfloor \frac{N}{k^2} \right\rfloor \text{ This is also the number of square-free numbers } \leq n$$

$$5. \textbf{Mobius inversion theorem:} \text{ The classic version states that if } g \text{ and } f \text{ are arithmetic functions satisfying } g(n) = \sum_{d|n} f(d) \text{ for every integer } n \geq 1 \text{ then } g(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right) \text{ for every integer } n \geq 1$$

$$6. \text{If } F(n) = \prod_{d|n} f(d), \text{ then } F(n) = \prod_{d|n} F\left(\frac{n}{d}\right)^{\mu(d)}$$

$$7. \sum_{d|n} \mu(d) \phi(d) = \prod_{j=1}^K (2 - P_j) \text{ where } p_j \text{ is the primes factorization of } d$$

8. If $F(n)$ is multiplicative, $F \not\equiv 0$, then

$$\sum_{d|n} \mu(d) f(d) = \prod_{i=1} (1 - f(P_i)) \cdot \text{where } p_i \text{ are primes of } n.$$

8.3.5 GCD and LCM

1. $\gcd(a, 0) = a$
2. $\gcd(a, b) = \gcd(b, a \bmod b)$
3. Every common divisor of a and b is a divisor of $\gcd(a, b)$.
4. if m is any integer, then $\gcd(a + m \cdot b, b) = \gcd(a, b)$
5. The gcd is a multiplicative function in the following sense:
if a_1 and a_2 are relatively prime, then
 $\gcd(a_1 \cdot a_2, b) = \gcd(a_1, b) \cdot \gcd(a_2, b)$.
6. $\gcd(a, b) \cdot \text{lcm}(a, b) = \|a \cdot b\|$
7. $\gcd(a, \text{lcm}(b, c)) = \text{lcm}(\gcd(a, b), \gcd(a, c))$.
8. $\text{lcm}(a, \gcd(b, c)) = \gcd(\text{lcm}(a, b), \text{lcm}(a, c))$.
9. For non-negative integers a and b , where a and b are not both zero, $\gcd(n^a - 1, n^b - 1) = n^{\gcd(a, b)} - 1$

$$10. \gcd(a, b) = \sum_{k \| a \text{ and } k \| b} \phi(k)$$

$$11. \sum_{i=1}^n [\gcd(i, n) = k] = \phi\left(\frac{n}{k}\right)$$

$$12. \sum_{k=1}^n \gcd(k, n) = \sum_{d \| n} d \cdot \phi\left(\frac{n}{d}\right)$$

$$13. \sum_{k=1}^n x^{\gcd(k, n)} = \sum_{d \| n} x^d \cdot \phi\left(\frac{n}{d}\right)$$

$$14. \sum_{k=1}^n \frac{1}{\gcd(k, n)} = \sum_{d \| n} \frac{1}{d} \cdot \phi\left(\frac{n}{d}\right) = \frac{1}{n} \sum_{d \| n} d \cdot \phi(d)$$

$$15. \sum_{k=1}^n \frac{k}{\gcd(k, n)} = \frac{n}{2} \cdot \sum_{d \| n} \frac{1}{d} \cdot \phi\left(\frac{n}{d}\right) = \frac{n}{2} \cdot \frac{1}{n} \cdot \sum_{d \| n} d \cdot \phi(d)$$

$$16. \sum_{k=1}^n \frac{n}{\gcd(k, n)} = 2 * \sum_{k=1}^n \frac{k}{\gcd(k, n)} - 1, \text{ for } n > 1$$

$$17. \sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] = \sum_{d=1}^n \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

$$18. \sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \phi(d) \left\lfloor \frac{n}{d} \right\rfloor^2$$

$$19. \sum_{i=1}^n \sum_{j=1}^n i \cdot j [\gcd(i, j) = 1] = \sum_{i=1}^n \phi(i) i^2$$

$$20. F(n) = \sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = \sum_{l=1}^n \left(\frac{(1 + \lfloor \frac{n}{l} \rfloor) (\lfloor \frac{n}{l} \rfloor)}{2} \right)^2 \sum_{d \| l} \mu(d) l d$$

$$21. \gcd(\text{lcm}(a, b), \text{lcm}(b, c), \text{lcm}(a, c)) = \text{lcm}(\gcd(a, b), \gcd(b, c), \gcd(a, c))$$

$$22. \gcd(A_L, A_{L+1}, \dots, A_R) = \gcd(A_L, A_{L+1} - A_L, \dots, A_R - A_{R-1}).$$

$$23. \text{ Given } n, \text{ If } SUM = LCM(1, n) + LCM(2, n) + \dots + LCM(n, n) \text{ then } SUM = \frac{n}{2} \left(\sum_{d|n} (\phi(d) \times d) + 1 \right)$$

8.3.6 Legendre Symbol

1. Let p be an odd prime number. An integer a is a quadratic residue modulo p if it is congruent to a perfect square modulo p and is a quadratic nonresidue modulo p otherwise. The Legendre symbol is a function of a and p defined as

$$\left(\frac{a}{p} \right) = \begin{cases} 1 & \text{if } a \text{ is a quadratic residue modulo } p \text{ and } a \not\equiv 0 \pmod{p} \\ -1 & \text{if } a \text{ is a non-quadratic residue modulo } p, \\ 0 & \text{if } a \equiv 0 \pmod{p} \end{cases}$$

2. Legendre's original definition was by means of explicit formula $\left(\frac{a}{p} \right) \equiv a^{\frac{p-1}{2}} \pmod{p}$ and $\left(\frac{a}{p} \right) \in \{-1, 0, 1\}$.

3. The Legendre symbol is periodic in its first (or top) argument: if $a \equiv b \pmod{p}$, then $\left(\frac{a}{p} \right) = \left(\frac{b}{p} \right)$.

4. . The Legendre symbol is a completely multiplicative function of its top argument: $\left(\frac{ab}{p} \right) = \left(\frac{a}{p} \right) \left(\frac{b}{p} \right)$

5. The Fibonacci numbers 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ... are defined by the recurrence $F_1 = F_2 = 1, F_{n+1} = F_n + F_{n-1}$. If p is a prime number then

$$F_{p-\left(\frac{p}{5}\right)} \equiv 0 \pmod{p}, \quad F_p \equiv \left(\frac{p}{5}\right) \pmod{p}.$$

$$\text{For example, } \left(\frac{2}{5}\right) = -1, \quad F_3 = 2, \quad F_2 = 1,$$

$$\left(\frac{3}{5}\right) = -1, \quad F_4 = 3, \quad F_3 = 2,$$

$$\left(\frac{5}{5}\right) = 0, \quad F_5 = 5,$$

$$\left(\frac{7}{5}\right) = -1, \quad F_8 = 21, \quad F_7 = 13,$$

$$\left(\frac{11}{5}\right) = 1, \quad F_{10} = 55, \quad F_{11} = 89,$$

6. Continuing from previous point, $\left(\frac{p}{5}\right) =$ infinite concatenation of the sequence (1, -1, -1, 1, 0) from $p \geq 1$.

7. If $n = k^2$ is perfect square then $\left(\frac{n}{p}\right) = 1$ for every odd prime except $\left(\frac{n}{k}\right) = 0$ if k is an odd prime.

8.4 Miscellaneous

1. $a + b = a \oplus b + 2(a \& b)$.
2. $a + b = a | b + a \& b$
3. $a \oplus b = a | b - a \& b$
4. k_{th} bit is set in x iff $x \bmod 2^{k-1} \geq 2^k$. It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.
5. k_{th} bit is set in x iff $x \bmod 2^{k-1} - x \bmod 2^k \neq 0$ ($= 2^k$ to be exact). It comes handy when you need to look at the bits of the numbers which are pair sums or subset sums etc.
6. $n \bmod 2^i = n \& (2^i - 1)$
7. $1 \oplus 2 \oplus 3 \oplus \dots \oplus (4k - 1) = 0$ for any $k \geq 0$
8. **Erdos Gallai Theorem:** The degree sequence of an undirected graph is the non-increasing sequence of its vertex degrees. A sequence of non-negative integers $d_1 \geq d_2 \geq \dots \geq d_n$ can be represented as the degree sequence of finite simple graph on n vertices if and only if $d_1 + d_2 + \dots + d_n$ is even and

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k)$$

holds for every k in $1 \leq k \leq n$.

Задача	Статус	Вася	Дима	Саша	Идеи
А - 1					
В - 2					
С - 3					
Д - 4					
Е - 5					
F - 6					
G - 7					
Н - 8					
I - 9					
J - 10					
К - 11					
L - 12					
М - 13					
N - 14					
О - 15					